DATA 260 – HW 6

7- Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b (seed 3319)

GitHub repository: https://github.com/villyalsriprajeet-cmd/data260-3319

Tagged commit hash (hw6): 835c514d1eef7b2babd15a8f1423bb48573dfaec

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

Part 1 - MongoDB Document API

1. Database

I made a MongoDB database called s3319_doc. It runs in Docker in a container called s3319_mongo. Inside it there is a fixtures collection, which is the document version of my main entity. When the app starts it creates the collection and adds a jsonSchema validator, so MongoDB itself refuses bad data even if something skips the API.

Requirement	Field	Rule
Required short text (max 100)	fixture_title	text, 1 to 100 characters, required
Optional description	description	text or empty
Categorical field with a default	status	Scheduled / Live / Completed / Postponed, default Scheduled
Categorical field with fixed options	category	League / Cup / Friendly / Tournament, required
Required date/time	kickoff_at	a real date, required
Automatic timestamps	created_at, updated_at	set by the server on create and on update

Connection and the fixtures validator (code/api/mongo_db.py)

code > api > mongo_db.py > ...

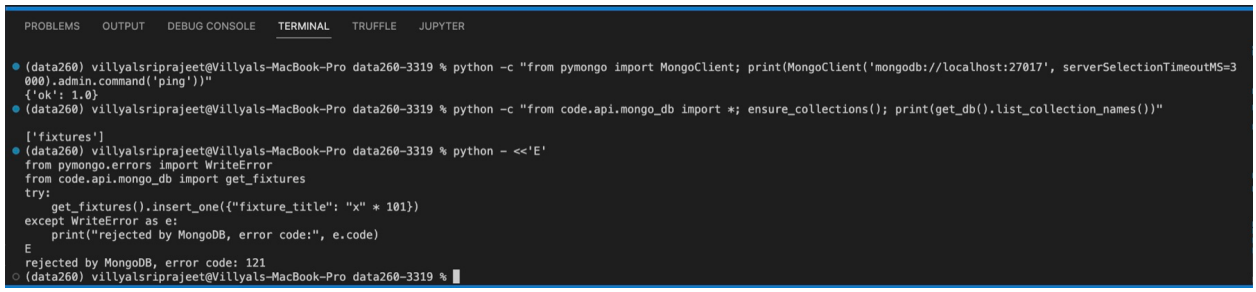
```python
# PyMongo connection, collection rules and indexes for the fixtures (Part 1) and memory (Part 2) collections
from pymongo import ASCENDING, MongoClient
from .config import (CATEGORY_VALUES, EPISODES_COLLECTION, FIXTURES_COLLECTION, MESSAGES_COLLECTION, MONGODB_DB,
                     MONGODB_URI, STATUS_VALUES, SUMMARIES_COLLECTION)
_client = None  # one shared client for the whole app
def get_client() -> MongoClient:
    global _client
    if _client is None:
        _client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=3000)  # fail fast if Mongo is down
    return _client
def get_db():
    return get_client()[MONGODB_DB]
def get_fixtures():
    return get_db()[FIXTURES_COLLECTION]
def get_messages():
    return get_db()[MESSAGES_COLLECTION]
def get_summaries():
    return get_db()[SUMMARIES_COLLECTION]
def get_episodes():
    return get_db()[EPISODES_COLLECTION]
FIXTURES_VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["fixture_title", "status", "category", "kickoff_at", "created_at", "updated_at"],  # mandatory fields
    "properties": {
        "fixture_title": {"bsonType": "string", "minLength": 1, "maxLength": 100},  # required short text
        "description": {"bsonType": ["string", "null"]},  # optional text
        "status": {"enum": STATUS_VALUES},  # constrained field with a default set by the API
        "category": {"enum": CATEGORY_VALUES},  # constrained field with a fixed set
        "kickoff_at": {"bsonType": "date"},  # required date/time
        "created_at": {"bsonType": "date"},
        "updated_at": {"bsonType": "date"},
    }}}
```

Creating the collections with their rules and indexes (code/api/mongo_db.py)

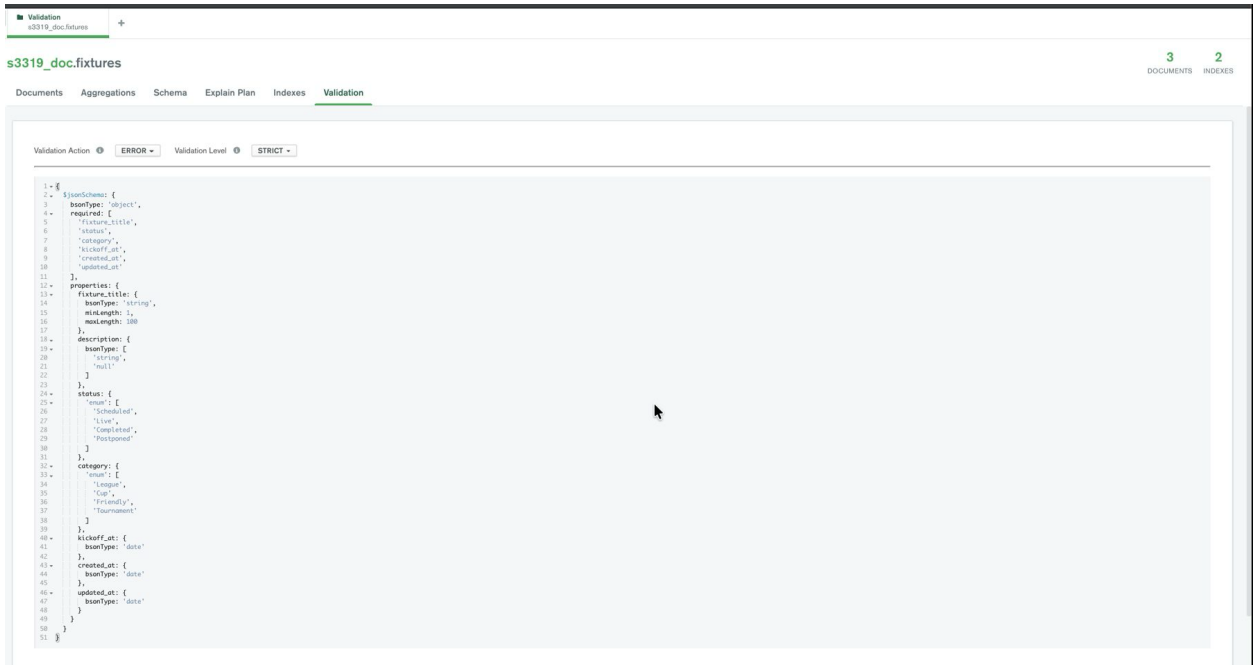
```python
COLLECTIONS = {
    FIXTURES_COLLECTION: (FIXTURES_VALIDATOR, [[("kickoff_at", ASCENDING)]]),
    MESSAGES_COLLECTION: (MESSAGES_VALIDATOR, [[("user_id", ASCENDING), ("session_id", ASCENDING), ("created_at", ASCENDING)],
                                               [("user_id", ASCENDING), ("created_at", ASCENDING)]]),
    SUMMARIES_COLLECTION: (SUMMARIES_VALIDATOR, [[("user_id", ASCENDING), ("scope", ASCENDING), ("created_at", ASCENDING)]]),
    EPISODES_COLLECTION: (EPISODES_VALIDATOR, [[("user_id", ASCENDING), ("created_at", ASCENDING)]]),
}
def ensure_collections():
    db = get_db()
    existing = db.list_collection_names()
    for name, (validator, indexes) in COLLECTIONS.items():
        if name not in existing:
            db.create_collection(name, validator=validator)  # create with rules
        else:
            db.command("collMod", name, validator=validator)  # refresh the rules
        for keys in indexes:
            db[name].create_index(keys)  # speed up the lookups the memory code does

```

Checking the connection, and MongoDB rejecting a 101-character title (error 121)



MongoDB Compass - the validation rules on s3319_doc.fixtures



2. API Implementation

I split the code the way the assignment asks. main.py sets up the server, the CORS middleware and the error handlers. mongo_db.py holds the connection. doc_schemas.py has the Pydantic models, and routers/docs.py has the routes. FastAPI turns the JSON body of each request into a Pydantic model before my code sees it. The app runs on port 8619.

Server setup, CORS and the error handlers (code/api/main.py)

code > api > main.py > add_sql_count

```python
# HW6 – Community Sports League Fixtures API (Domain 7): MySQL fixtures and teams from HW5 plus the MongoDB document API, port 8619
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from pymongo.errors import PyMongoError, ServerSelectionTimeoutError, WriteError
from fastapi.middleware.cors import CORSMiddleware
from .routers.auth import router as auth_router
from .routers.fixtures import router as fixtures_router
from .routers.teams import router as teams_router
from .routers.docs import router as docs_router
from .routers.chat import router as chat_router
from .mongo_db import ensure_collections
from .perf import sql_counter
PORT_BASE = 8619
@asynccontextmanager
async def lifespan(app):
    try:
        ensure_collections()  # create the Mongo collections and rules on startup
    except PyMongoError as exc:
        print(f"WARNING: MongoDB not ready: {exc}")
    yield
app = FastAPI(title="Community Sports League Fixtures – HW6", lifespan=lifespan)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],  # the React dev server
    allow_credentials=True,  # allow the session cookie on cross origin calls
    allow_methods=["*"],
    allow_headers=["*"],
)
@app.middleware("http")
async def add_sql_count(request, call_next):
    box = {"n": 0}
    sql_counter.set(box)
    response = await call_next(request)
    response.headers["X-SQL-Count"] = str(box["n"])  # statements this request ran
    return response
app.include_router(auth_router)  # POST /login
app.include_router(fixtures_router)  # CRUD on /fixtures
app.include_router(teams_router)  # CRUD on /teams and /teams/{id}/fixtures
app.include_router(docs_router)  # HW6 Part 1: CRUD on /api/fixtures (MongoDB)
app.include_router(chat_router)  # HW6 Part 2: /api/chat, /api/memory, /api/aggregate
@app.exception_handler(WriteError)
async def mongo_write_error(request, exc):
    code = 422 if exc.code == 121 else 500  # 121 = the collection validator rejected the document
    return JSONResponse(status_code=code, content={"detail": f"Database rejected the write: {exc}"})
@app.exception_handler(ServerSelectionTimeoutError)
async def mongo_down(request, exc):
    return JSONResponse(status_code=503, content={"detail": "MongoDB is not reachable"})
@app.exception_handler(PyMongoError)
async def mongo_error(request, exc):
    return JSONResponse(status_code=500, content={"detail": "Database error"})
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=PORT_BASE)

```

The five CRUD routes (code/api/routers/docs.py)

code > api > routers > docs.py > serialize

```python
# Part 1 CRUD routes for the MongoDB fixtures collection
from datetime import datetime, timezone
from bson import ObjectId
from bson.errors import InvalidId
from fastapi import APIRouter, HTTPException, Query
from pymongo import ReturnDocument
from ..doc_schemas import FixtureDocCreate, FixtureDocOut, FixtureDocUpdate
from ..mongo_db import get_fixtures
router = APIRouter(prefix="/api/fixtures", tags=["Fixtures (MongoDB)"])
def to_object_id(value: str) -> ObjectId:
    try:
        return ObjectId(value)
    except (InvalidId, TypeError):
        raise HTTPException(status_code=400, detail=f"Invalid id format: {value}")
def serialize(doc: dict) -> dict:
    doc["id"] = str(doc.pop("_id"))
    return doc
@router.post("", response_model=FixtureDocOut, status_code=201)
def create_fixture(payload: FixtureDocCreate):
    now = datetime.now(timezone.utc)
    doc = {**payload.model_dump(), "created_at": now, "updated_at": now}  # timestamps set by the server
    doc["_id"] = get_fixtures().insert_one(doc).inserted_id
    return serialize(doc)
@router.get("", response_model=list[FixtureDocOut])
def list_fixtures(skip: int = Query(0, ge=0), limit: int = Query(100, ge=1, le=500)):
    cursor = get_fixtures().find().sort("created_at", -1).skip(skip).limit(limit)  # newest first
    return [serialize(d) for d in cursor]
@router.get("/{fixture_id}", response_model=FixtureDocOut)
def get_fixture(fixture_id: str):
    doc = get_fixtures().find_one({"_id": to_object_id(fixture_id)})
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return serialize(doc)
@router.put("/{fixture_id}", response_model=FixtureDocOut)
def update_fixture(fixture_id: str, payload: FixtureDocUpdate):
    oid = to_object_id(fixture_id)
    changes = payload.model_dump(exclude_unset=True)  # only the fields that were sent
    if not changes:
        raise HTTPException(status_code=400, detail="No fields provided to update")
    changes["updated_at"] = datetime.now(timezone.utc)
    doc = get_fixtures().find_one_and_update({"_id": oid}, {"$set": changes}, return_document=ReturnDocument.AFTER)
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return serialize(doc)
@router.delete("/{fixture_id}")
def delete_fixture(fixture_id: str):
    if get_fixtures().delete_one({"_id": to_object_id(fixture_id)}).deleted_count == 0:
        raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
    return {"message": "Fixture deleted", "id": fixture_id}

```

Terminal outputs

POST /api/fixtures three times - 201 Created (status is Scheduled when I don't send it)



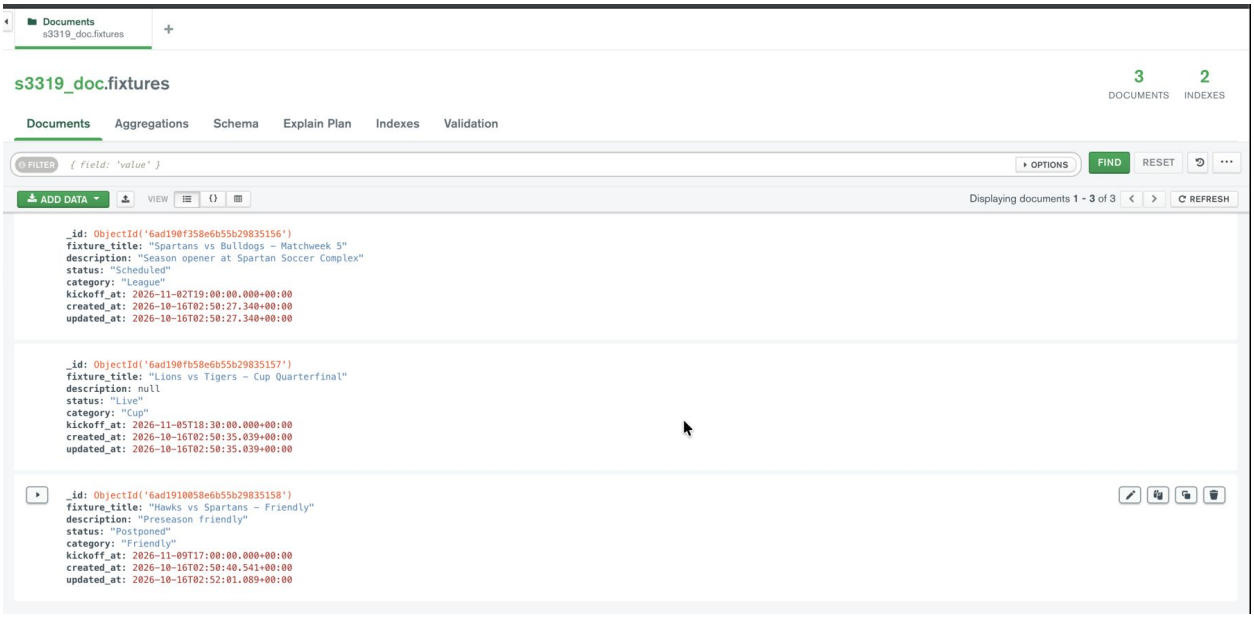
GET all - 200, GET by id - 200, PUT {status: Postponed} - 200 (only status and updated_at change)



Create a temporary fixture, DELETE it - 200, then GET it again - 404 Not Found

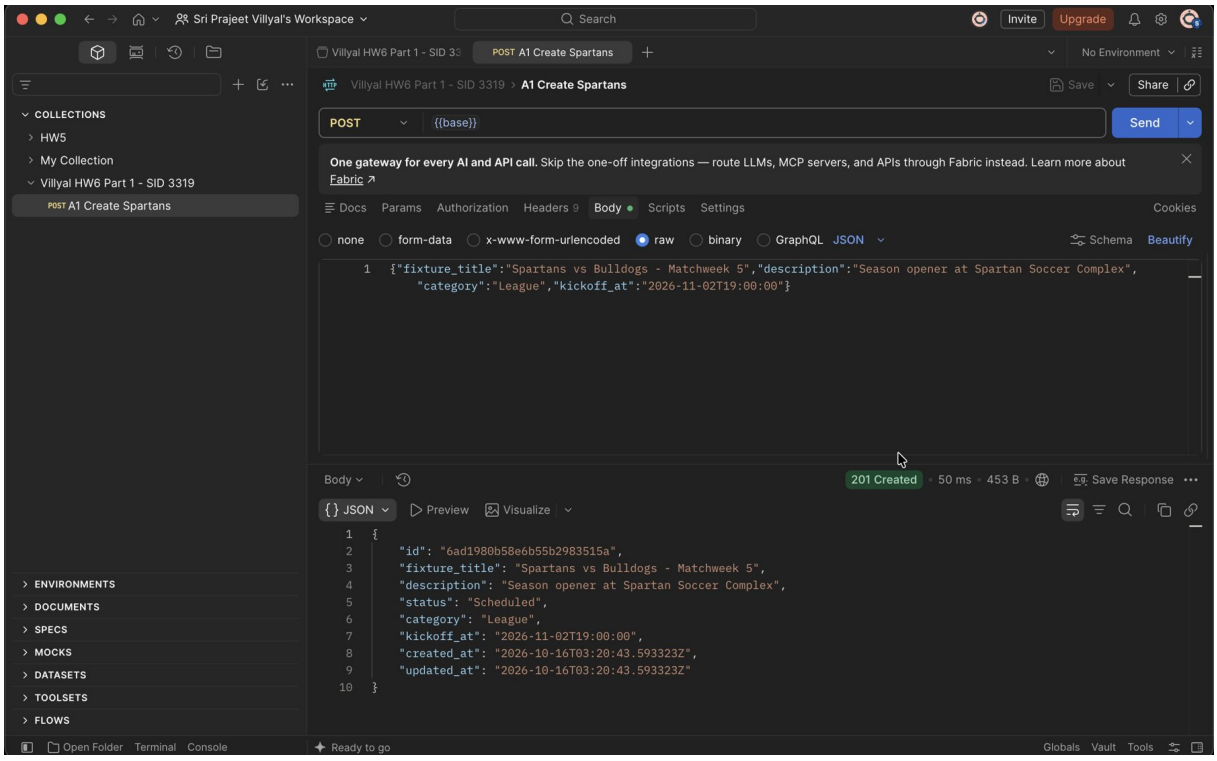


MongoDB Compass - the same three fixtures (same ids), with status changed to Postponed by the PUT

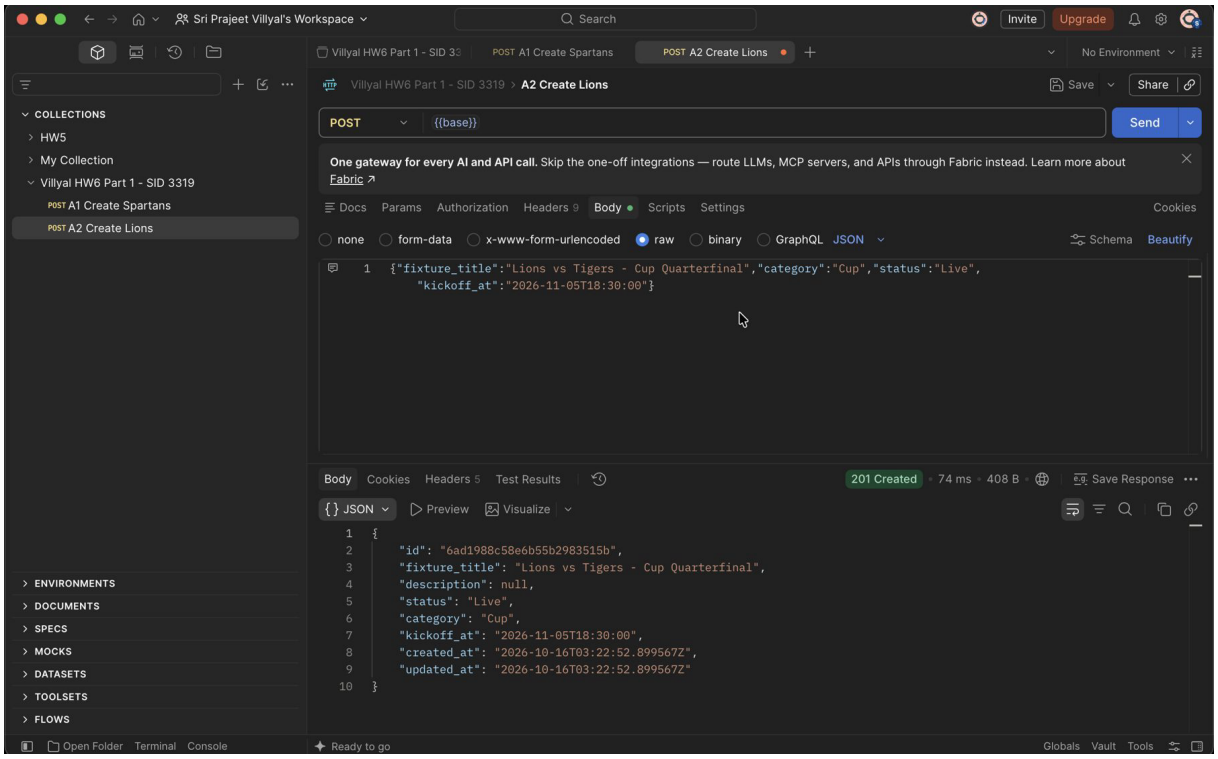


Postman outputs (the same requests repeated in Postman)

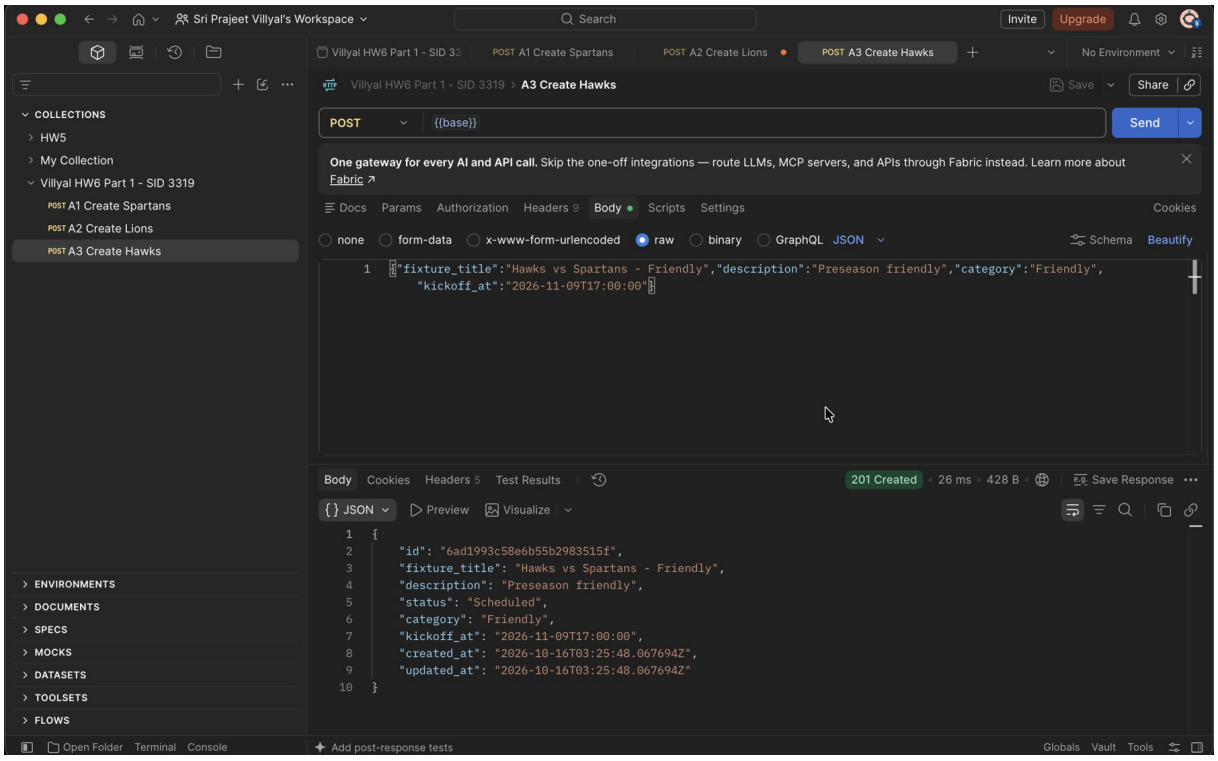
POST /api/fixtures - 201 Created (status defaults to Scheduled)



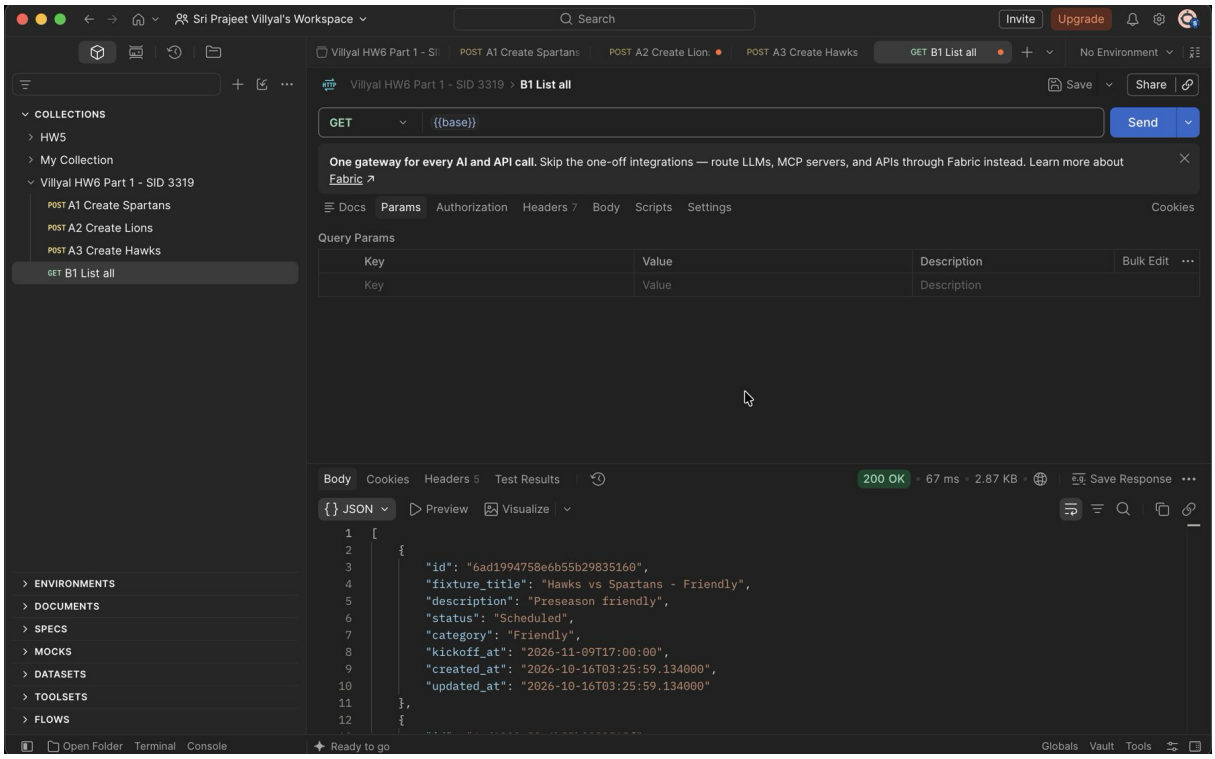
POST /api/fixtures - 201 Created (Cup fixture, status Live)



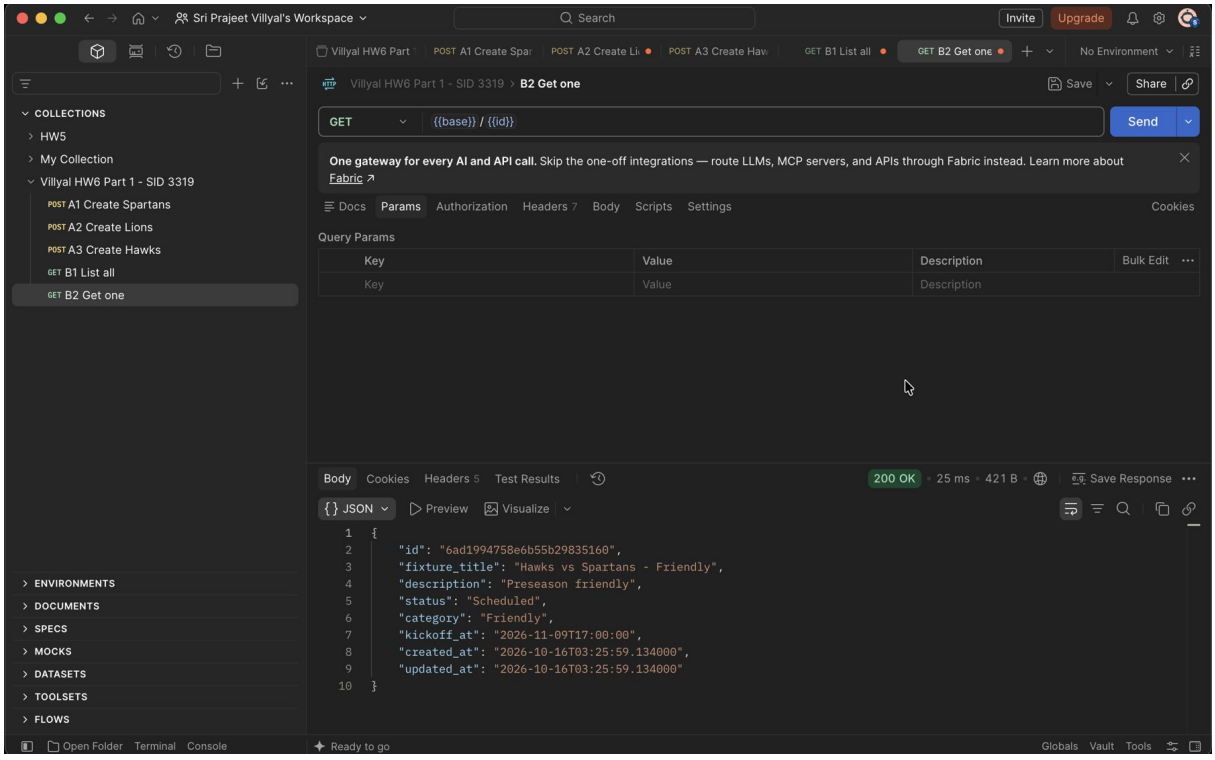
POST /api/fixtures - 201 Created (Friendly fixture)



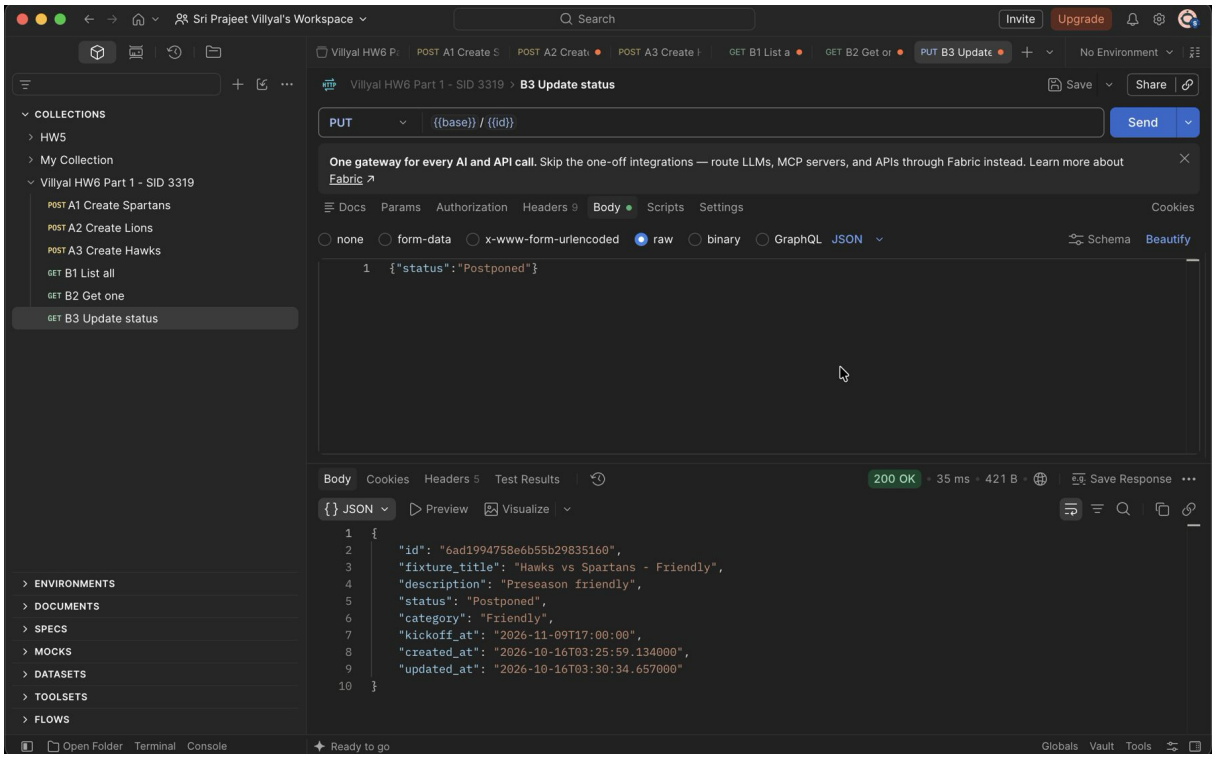
GET /api/fixtures - 200 OK, newest first



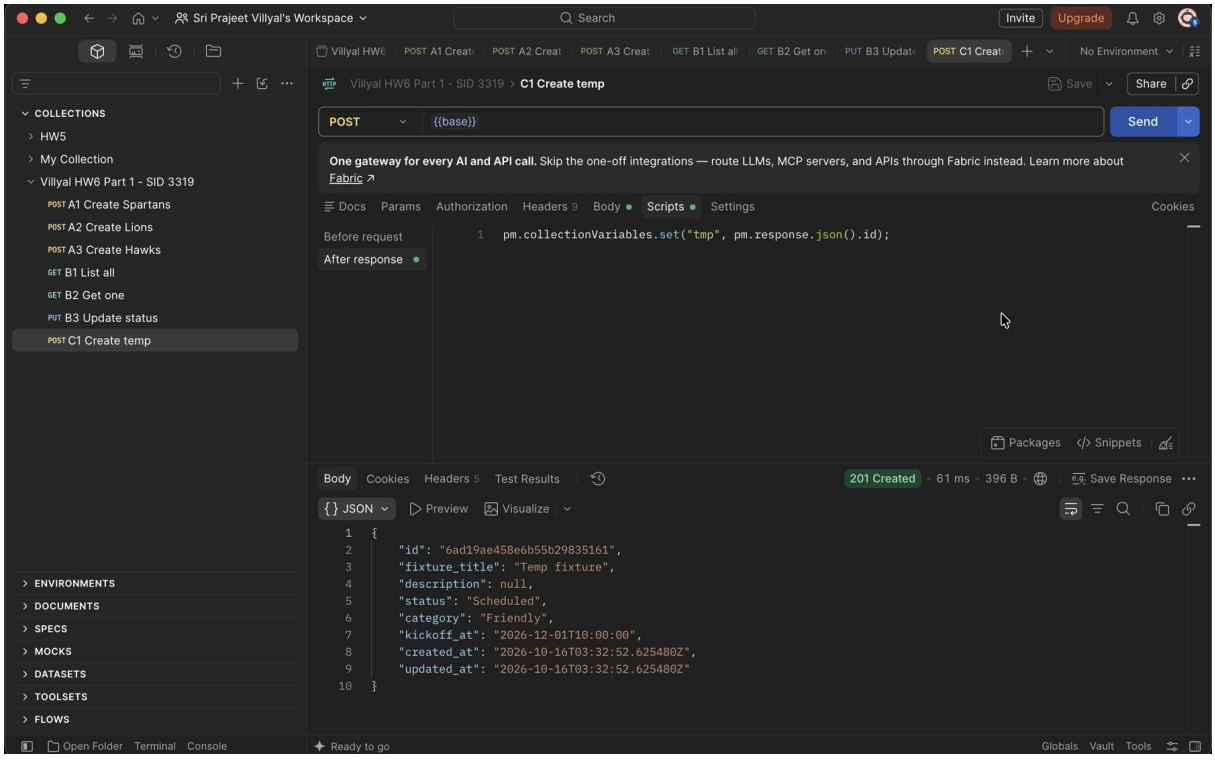
GET /api/fixtures/{id} - 200 OK



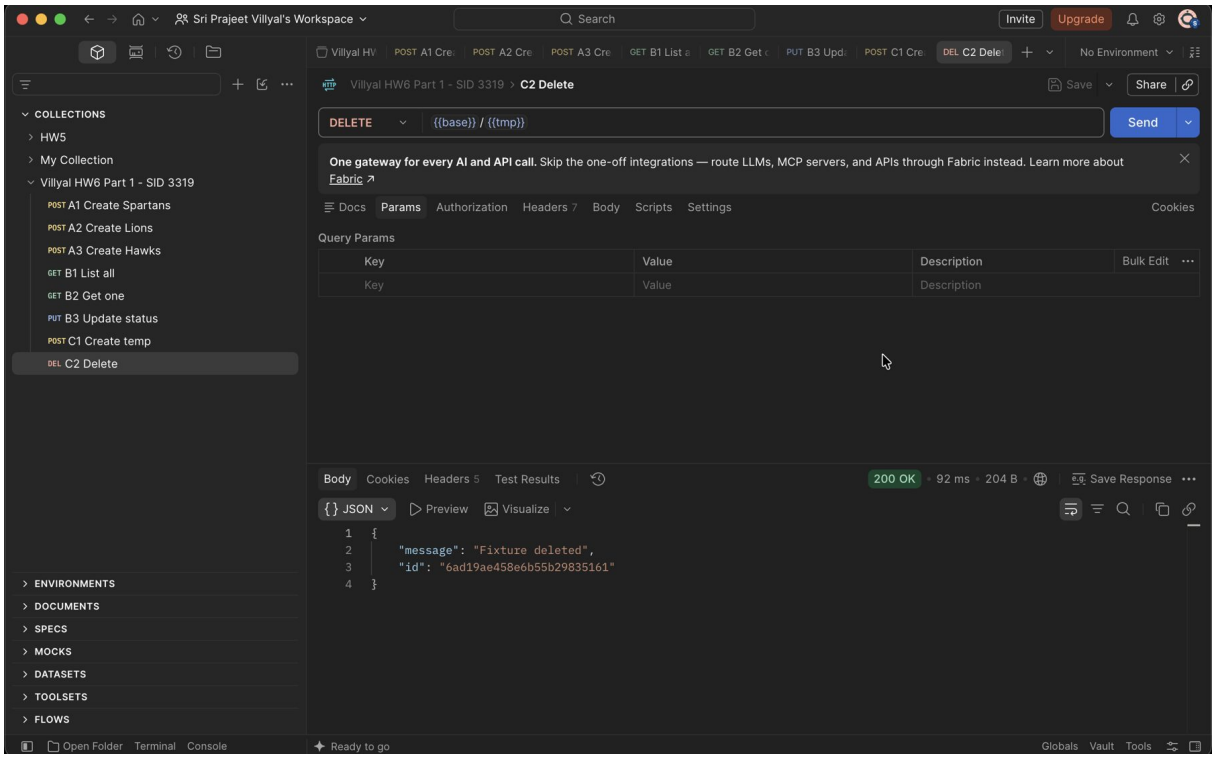
PUT /api/fixtures/{id}{status: Postponed} - 200 OK



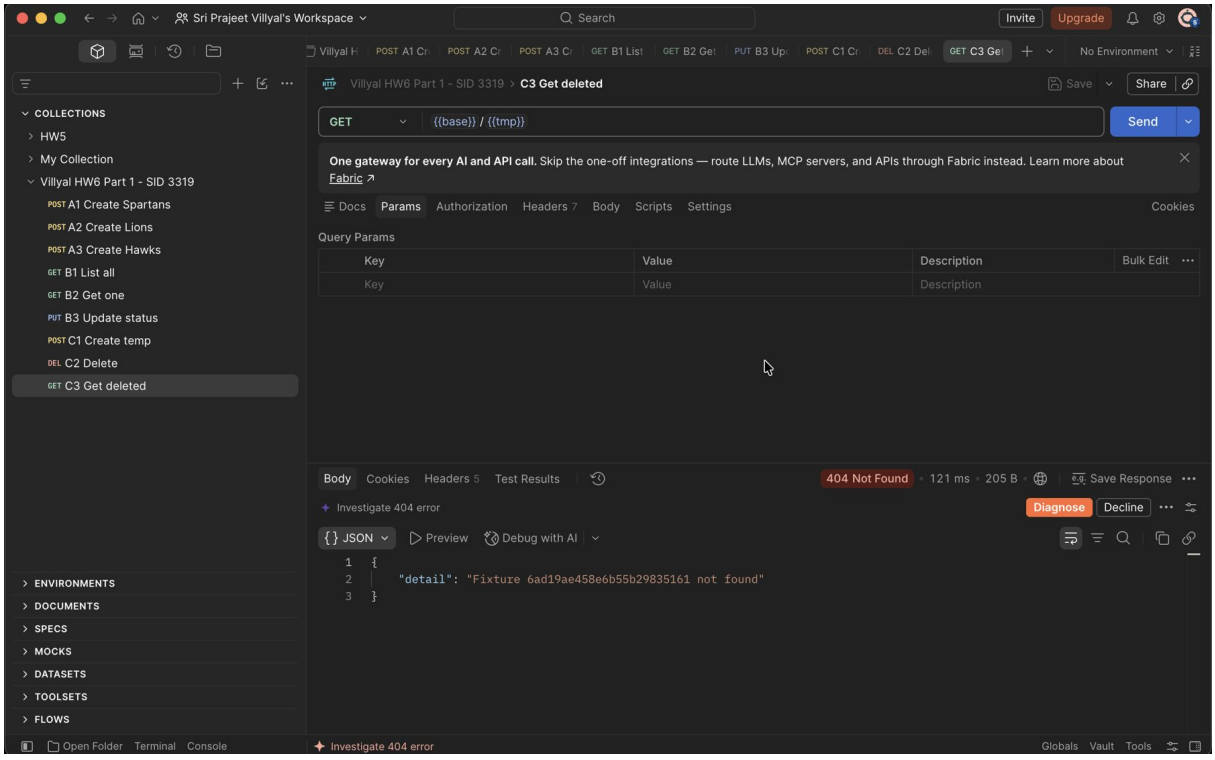
POST a temporary fixture - 201 Created



DELETE /api/fixtures/{id} - 200 OK



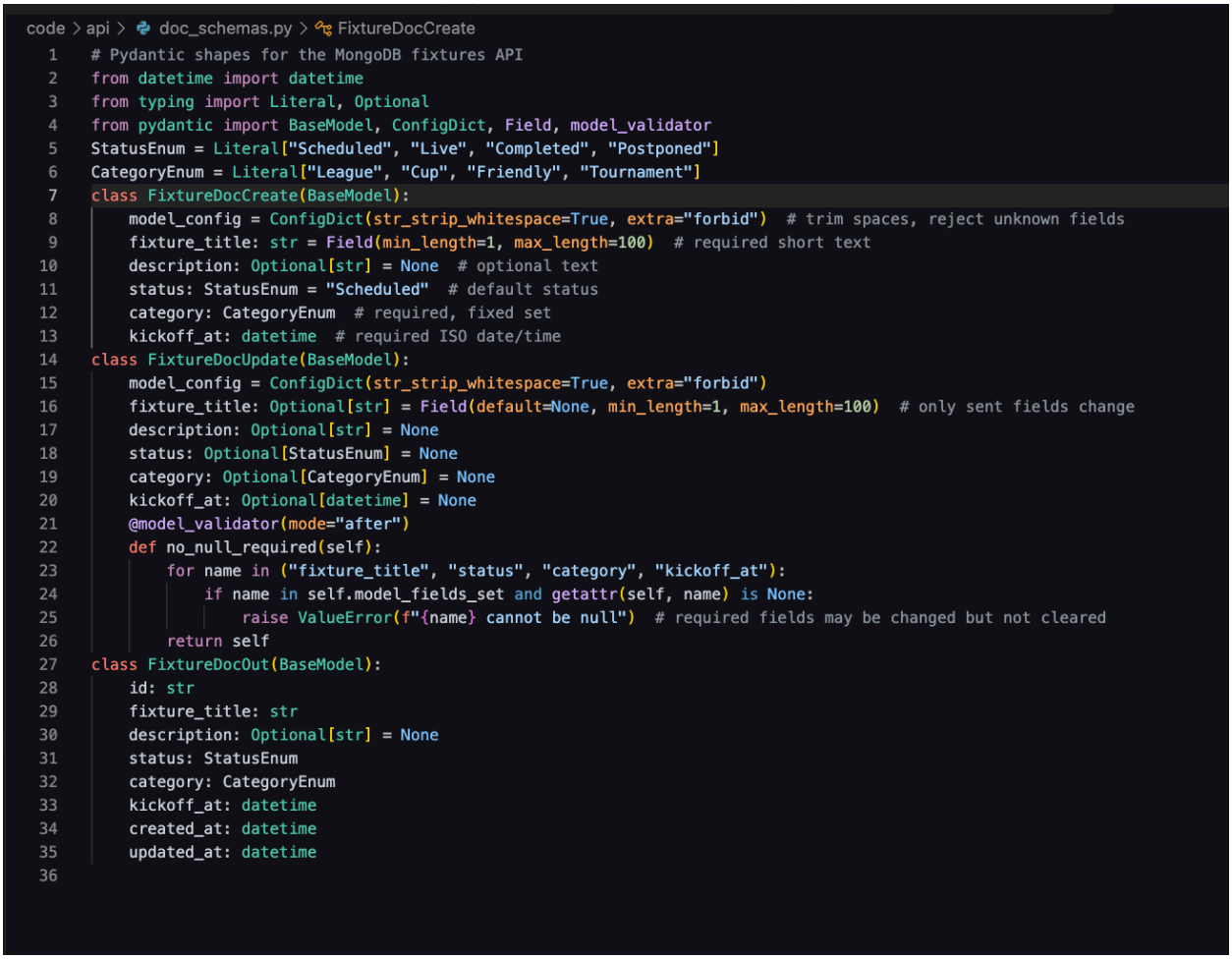
GET /api/fixtures/{id} after the delete - 404 Not Found



3. Data Validation

Pydantic checks every request before it gets to MongoDB. It makes sure the required fields are there, the title is not too long, status and category use allowed values, the date is a real date, and each field has the right type. It also trims spaces and rejects fields it doesn't know. On an update only the fields I send are changed, and a required field can't be set to empty.

Pydantic models for create, update and output (code/api/doc_schemas.py)



Bad data with curl: missing title, wrong status, bad date, wrong type - 422; a malformed id - 400

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % curl -s -w "\nHTTP %{http_code}\n" -X POST $B -H "$H" -d '{"category":"Cup","kickoff_at":"2026-11-02T19:00:00"}'

{"detail":[{"type":"missing","loc":["body","fixture_title"],"msg":"Field required","input":{"category":"Cup","kickoff_at":"2026-11-02T19:00:00"}}]}
HTTP 422
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % curl -s -w "\nHTTP %{http_code}\n" -X POST $B -H "$H" -d '{"fixture_title":"Bad enum","status":"Cancelled","category"
:"Cup","kickoff_at":"2026-11-02T19:00:00"}'

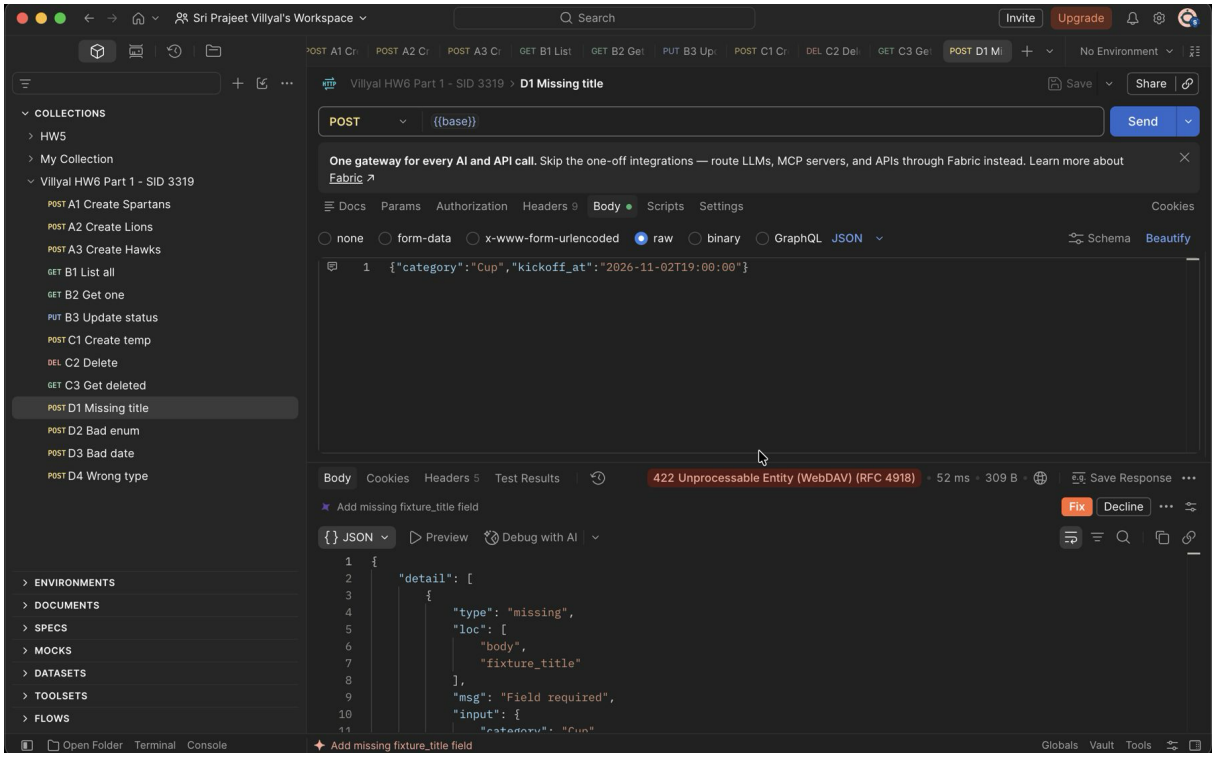
{"detail":[{"type":"literal_error","loc":["body","status"],"msg":"Input should be 'Scheduled', 'Live', 'Completed' or 'Postponed'","input":"Cancelled","ctx":{"expected":"'Scheduled'
, 'Live', 'Completed' or 'Postponed'"}}]}
HTTP 422
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % curl -s -w "\nHTTP %{http_code}\n" -X POST $B -H "$H" -d '{"fixture_title":"Bad date","category":"Cup","kickoff_at":"
tomorrow"}'

{"detail":[{"type":"datetime_from_date_parsing","loc":["body","kickoff_at"],"msg":"Input should be a valid datetime or date, input is too short","input":"tomorrow","ctx":{"error":"i
nput is too short"}}]}
HTTP 422
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % curl -s -w "\nHTTP %{http_code}\n" -X POST $B -H "$H" -d '{"fixture_title":123,"category":"Cup","kickoff_at":"2026-11
-02T19:00:00"}'

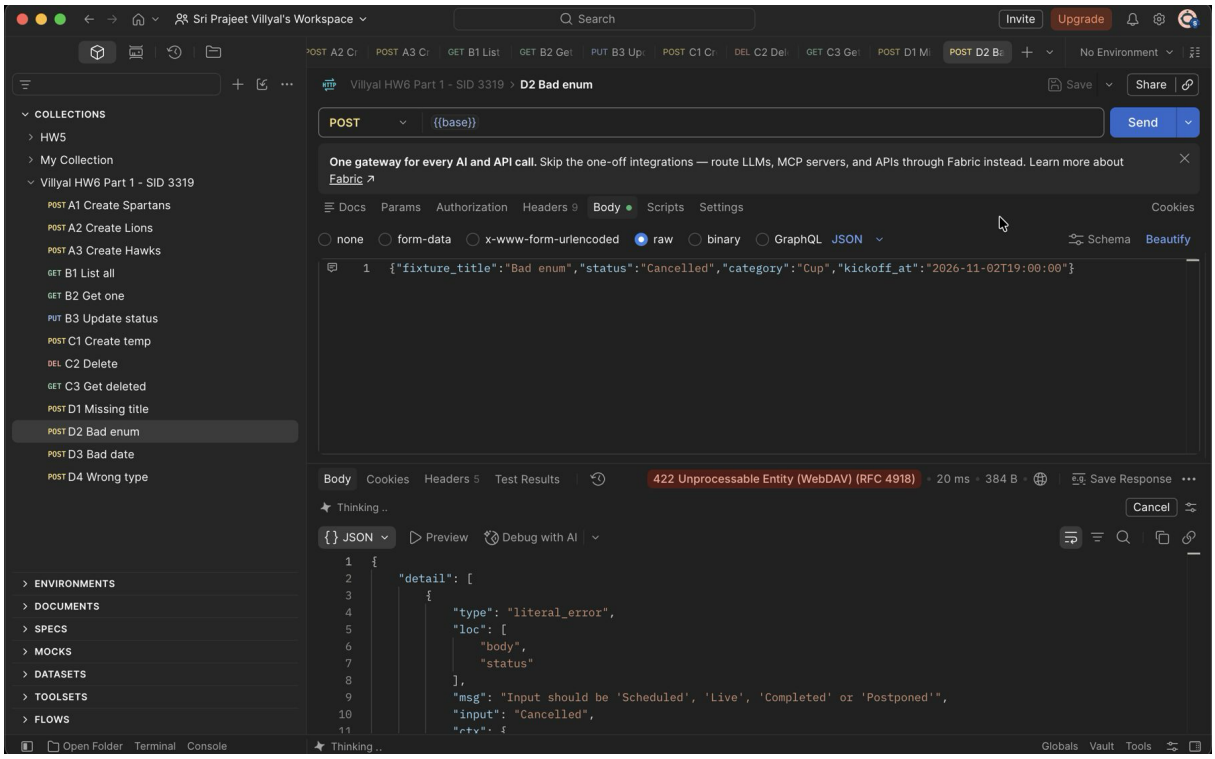
{"detail":[{"type":"string_type","loc":["body","fixture_title"],"msg":"Input should be a valid string","input":123}]}
HTTP 422
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % curl -s -w "\nHTTP %{http_code}\n" $B/not-an-id
{"detail":"Invalid id format: not-an-id"}
HTTP 400
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

Postman outputs (the same bad requests)

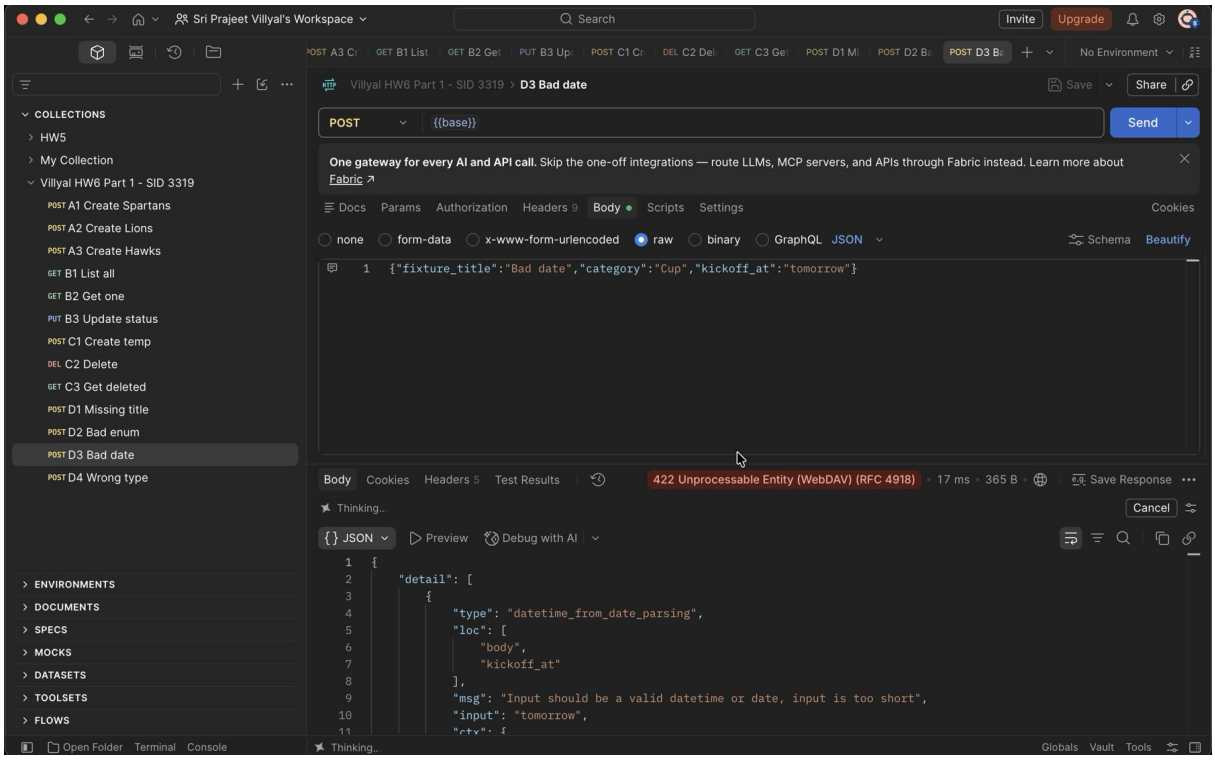
POST with no fixture_title - 422, "Field required"



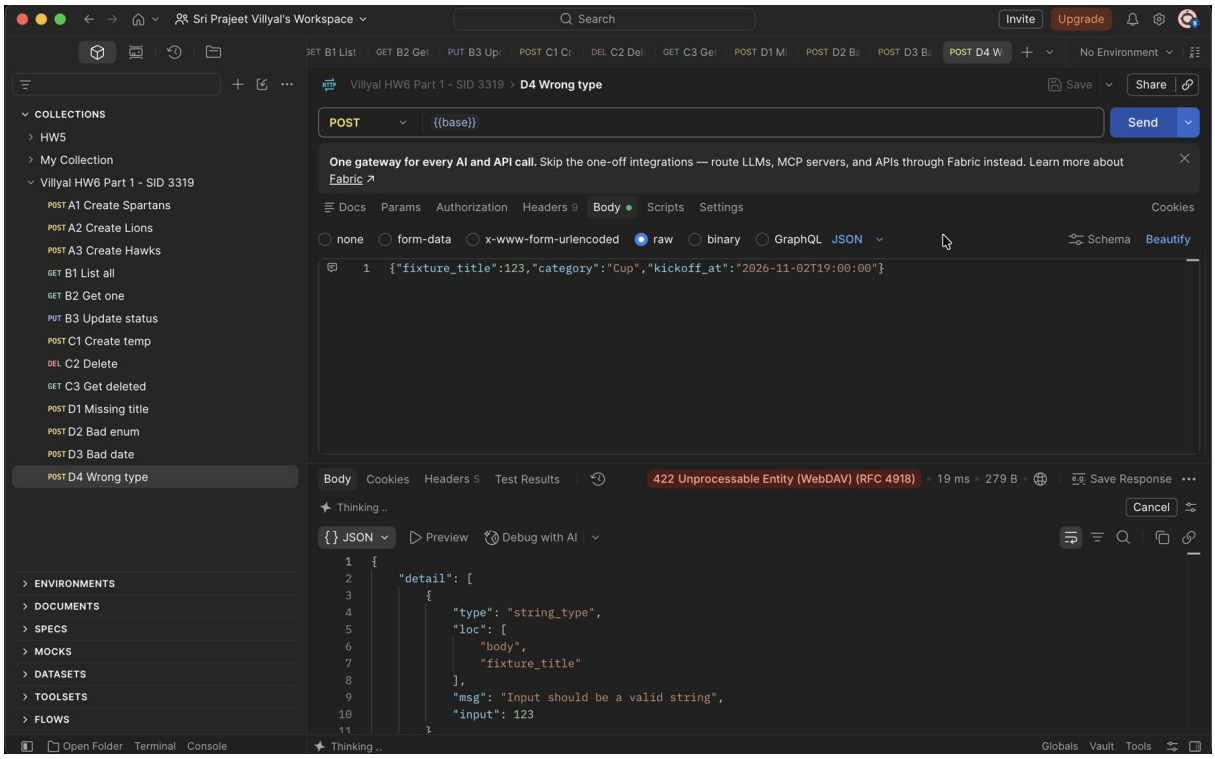
POST with status "Cancelled" - 422, it lists the allowed values



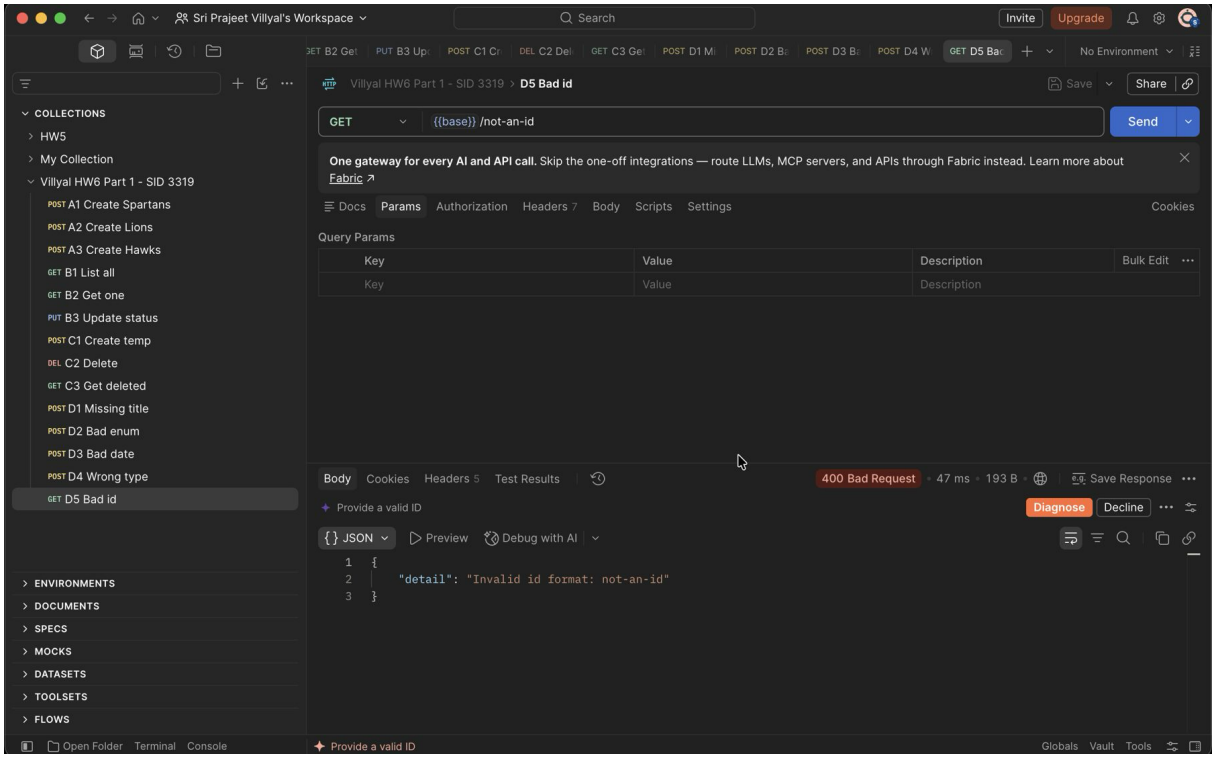
POST with kickoff_at "tomorrow" - 422, not a valid date



POST with fixture_title 123 - 422, "Input should be a valid string"



GET /api/fixtures/not-an-id - 400, "Invalid id format"



What went wrong	Status code	Caught by
Missing field, wrong value, bad date, wrong type	422	Pydantic
Malformed id	400	to_object_id in docs.py
Id not found	404	the route
MongoDB's own validator rejects the write	422	WriteError handler (error 121)
MongoDB down, or another database error	503 / 500	the handlers in main.py

Part 2 - Short-, Long-, and Episodic Memory

Part 2 uses the same database and connection and adds three new collections. The fixtures collection from Part 1 is left as it is. Each new collection has its own validator and indexes on user_id, session_id and created_at, so lookups stay quick.

Collection	What each document holds
messages	user_id, session_id, role (user or assistant), content, created_at
summaries	user_id, session_id (empty for the lifetime summary), scope ('session' or 'user'), text, created_at
episodes	user_id, session_id, a short fact, importance (0 to 1), its embedding (384 numbers), created_at

My settings are: the last 12 messages as short term memory, a session summary every 5 user messages, a lifetime summary after every 3 session summaries, up to 3 facts saved per message, and the top 3 facts retrieved each turn. Embeddings come from all-MiniLM-L6-v2, the same model I used in HW4, and the chat model is called through my HW1 adapter, src/model_client.py.

Validators for messages, summaries and episodes (code/api/mongo_db.py)

```
33  MESSAGES_VALIDATOR = {"$jsonSchema": {
34      "bsonType": "object",
35      "required": ["user_id", "session_id", "role", "content", "created_at"],
36      "properties": {
37          "user_id": {"bsonType": "string"},
38          "session_id": {"bsonType": "string"},
39          "role": {"enum": ["user", "assistant"]},
40          "content": {"bsonType": "string"},
41          "created_at": {"bsonType": "date"},
42      }}}
43  SUMMARIES_VALIDATOR = {"$jsonSchema": {
44      "bsonType": "object",
45      "required": ["user_id", "session_id", "scope", "text", "created_at"],
46      "properties": {
47          "user_id": {"bsonType": "string"},
48          "session_id": {"bsonType": ["string", "null"]},  # null for the lifetime summary
49          "scope": {"enum": ["session", "user"]},
50          "text": {"bsonType": "string"},
51          "created_at": {"bsonType": "date"},
52      }}}
53  EPISODES_VALIDATOR = {"$jsonSchema": {
54      "bsonType": "object",
55      "required": ["user_id", "session_id", "fact", "importance", "embedding", "created_at"],
56      "properties": {
57          "user_id": {"bsonType": "string"},
58          "session_id": {"bsonType": "string"},
59          "fact": {"bsonType": "string", "minLength": 1, "maxLength": 300},
60          "importance": {"bsonType": ["double", "int"], "minimum": 0, "maximum": 1},
61          "embedding": {"bsonType": "array", "items": {"bsonType": "double"}},
62          "created_at": {"bsonType": "date"},
63      }}}
```

1. POST /api/chat

The request has a user_id, an optional session_id and the message. If there is no session_id, it carries on with the user's latest session or starts a new one. Each turn follows the steps from the assignment in the same order, and the response returns the reply along with what memory was used: how many recent messages, the summary text and the facts that were pulled in.

The /api/chat endpoint (code/api/routers/chat.py)

code > api > routers > chat.py > ...

```python
# Part 2 endpoints: POST /api/chat, GET /api/memory/{user_id}, GET /api/aggregate/{user_id}
import uuid
from urllib.error import URLError
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, ConfigDict, Field
from .. import memory
from ..mongo_db import get_episodes, get_messages, get_summaries
router = APIRouter(prefix="/api", tags=["Chat memory (MongoDB)"])
class ChatRequest(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    user_id: str = Field(min_length=1, max_length=64)
    session_id: str | None = Field(default=None, min_length=1, max_length=64)  # optional, defaults to the latest session
    message: str = Field(min_length=1, max_length=2000)
def clean(doc):
    doc = {k: v for k, v in doc.items() if k not in ("_id", "embedding")}  # no ObjectId and no 384-number vectors in the JSON
    if doc.get("created_at"):
        doc["created_at"] = doc["created_at"].isoformat()
    return doc
def latest_session_id(user_id):
    doc = get_messages().find_one({"user_id": user_id}, sort=[("created_at", -1), ("_id", -1)])
    return doc["session_id"] if doc else None
@router.post("/chat")
def chat(body: ChatRequest):
    session_id = body.session_id or latest_session_id(body.user_id) or f"sess-{uuid.uuid4().hex[:8]}"  # latest session, else a new one
    try:
        out = memory.chat_turn(body.user_id, session_id, body.message)
    except (URLError, ConnectionError) as exc:
        raise HTTPException(status_code=503, detail=f"Model server is not reachable: {exc}")
    return {"user_id": body.user_id, "session_id": session_id, "reply": out["reply"], "used": out["used"], "metrics": out["metrics"]}
```

One chat turn - steps 1 to 9 are marked in the comments (code/api/memory.py)

```python
def chat_turn(user_id, session_id, message, short_term_n=None, summarize_every=None, top_k=None, log_path=config.METRICS_LOG, policy=None):
    n = config.SHORT_TERM_N if short_term_n is None else short_term_n
    every = config.SUMMARIZE_EVERY_USER_MSGS if summarize_every is None else summarize_every
    k = config.EPISODIC_TOP_K if top_k is None else top_k
    started = time.perf_counter()
    saved = save_message(user_id, session_id, "user", message)  # 1. save the user message
    window = short_term(user_id, session_id, n, saved["_id"])  # 2. short term window
    if policy == "evict_assistant":
        window = compress_window(window)  # eviction policy: drop old assistant replies
    session_summary = latest_summary(user_id, "session", session_id)  # 3. summaries
    lifetime_summary = latest_summary(user_id, "user")
    facts = retrieve_episodes(user_id, embed(message), k)  # 4. top-k episodic facts
    messages, parts = build_prompt(session_summary, lifetime_summary, window, message, facts)  # 5. prompt
    t0 = time.perf_counter()
    result = call_model(messages)  # 6. model call
    reply_ms = round((time.perf_counter() - t0) * 1000)
    reply = result["text"].strip() or "(empty reply)"
    save_message(user_id, session_id, "assistant", reply)  # 7. save the reply
    new_facts, extract = extract_facts(message)  # 8. episodic extraction
    saved_facts = store_episodes(user_id, session_id, new_facts)
    user_count = get_messages().count_documents({"user_id": user_id, "session_id": session_id, "role": "user"})
    summary = None
    if every > 0 and user_count % every == 0:  # 9. summary every M user messages
        summary = summarize_session(user_id, session_id)
        lifetime = maybe_lifetime(user_id)
        if lifetime:
            summary = {"input_tokens": summary["input_tokens"] + lifetime["input_tokens"],
                       "output_tokens": summary["output_tokens"] + lifetime["output_tokens"]}
    metrics = {"timestamp": now().isoformat(timespec="seconds"), "user_id": user_id, "session_id": session_id, "turn": user_count,
               "short_term_n": n, "prompt_tokens": result["input_tokens"], "completion_tokens": result["output_tokens"],
               "primer_tokens": count_tokens(parts["primer"]), "long_term_tokens": count_tokens(parts["long_term"]),
               "short_term_tokens": count_tokens(parts["short_term"]), "current_tokens": count_tokens(parts["current"]),
               "episodic_tokens": count_tokens(parts["episodic"]), "extract_prompt_tokens": extract["input_tokens"],
               "extract_completion_tokens": extract["output_tokens"],
               "summary_prompt_tokens": summary["input_tokens"] if summary else 0,
               "summary_completion_tokens": summary["output_tokens"] if summary else 0,
               "latency_ms": round((time.perf_counter() - started) * 1000), "reply_latency_ms": reply_ms,
               "token_method": token_method()}
    if log_path:
        log_metrics(metrics, log_path)
    used = {"short_term_count": len(window), "short_term": window, "long_term_summary": parts["long_term"] or None,
            "episodic_facts": facts, "new_facts_saved": [r["fact"] for r in saved_facts], "summary_created": summary is not None}
    return {"reply": reply, "used": used, "metrics": metrics}

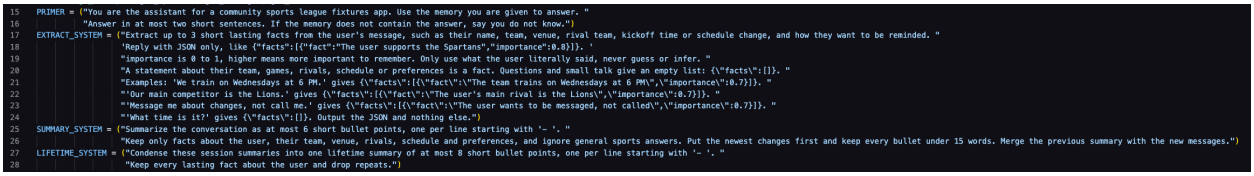
```

2. Memory logic

Short-term window and loading the latest summaries

```
def short_term(user_id, session_id, n, before_id=None):
    if n <= 0:
        return []  # limit(0) would mean no limit in MongoDB
    query = {"user_id": user_id, "session_id": session_id}
    if before_id is not None:
        query["_id"] = {"$lt": before_id}  # only messages saved before the current one
    rows = list(get_messages().find(query).sort([("created_at", -1), ("_id", -1)]).limit(n))  # newest N
    return [{"role": r["role"], "content": r["content"]} for r in reversed(rows)]  # back to oldest first
def latest_summary(user_id, scope, session_id=None):
    query = {"user_id": user_id, "scope": scope}
    if scope == "session":
        query["session_id"] = session_id
    doc = get_summaries().find_one(query, sort=[("created_at", -1), ("_id", -1)])
    return doc["text"] if doc else None
```

The system primer and the prompts for fact extraction and summaries



```
PRIMER = ("You are the assistant for a community sports league fixtures app. Use the memory you are given to answer. "
          "Answer in at most two short sentences. If the memory does not contain the answer, say you do not know.")
EXTRACT_SYSTEM = ("Extract up to 3 short lasting facts from the user's message, such as their name, team, venue, rival team, kickoff time or schedule change, and how they want to be reminded. "
                  'Reply with JSON only, like {"facts":[{"fact":"The user supports the Spartans","importance":0.8}]}. '
                  "importance is 0 to 1, higher means more important to remember. Only use what the user literally said, never guess or infer. "
                  "A statement about their team, games, rivals, schedule or preferences is a fact. Questions and small talk give an empty list: {\"facts\":[]}. "
                  "Examples: 'We train on Wednesdays at 6 PM.' gives {\"facts\":[{\"fact\":\"The team trains on Wednesdays at 6 PM\",\"importance\":0.7}]}. "
                  "'Our main competitor is the Lions.' gives {\"facts\":[{\"fact\":\"The user's main rival is the Lions\",\"importance\":0.7}]}. "
                  "'Message me about changes, not call me.' gives {\"facts\":[{\"fact\":\"The user wants to be messaged, not called\",\"importance\":0.7}]}. "
                  "'What time is it?' gives {\"facts\":[]}. Output the JSON and nothing else.")
SUMMARY_SYSTEM = ("Summarize the conversation as at most 6 short bullet points, one per line starting with '- '. "
                  "Keep only facts about the user, their team, venue, rivals, schedule and preferences, and ignore general sports answers. Put the newest changes first and keep every bullet under 15 words. Merge the previous summary with the new messages.")
LIFETIME_SYSTEM = ("Condense these session summaries into one lifetime summary of at most 8 short bullet points, one per line starting with '- '. "
                   "Keep every lasting fact about the user and drop repeats.")
```

Pulling facts out of a message, saving them with embeddings, and finding the closest ones by cosine similarity

```
def extract_facts(message):
    result = call_model([{"role": "system", "content": EXTRACT_SYSTEM}, {"role": "user", "content": message}],
                        json_mode=True, num_predict=120)
    return parse_facts(result["text"]), result
def store_episodes(user_id, session_id, facts):
    known = {d["fact"].lower() for d in get_episodes().find({"user_id": user_id}, {"fact": 1})}
    rows = []
    for f in facts:
        if f["fact"].lower() in known:
            continue  # this fact is already stored
        known.add(f["fact"].lower())
        rows.append({"user_id": user_id, "session_id": session_id, "fact": f["fact"], "importance": f["importance"],
                     "embedding": embed(f["fact"]), "created_at": now()})  # embedding stored as an array of floats
    if rows:
        get_episodes().insert_many(rows)
    return rows
def retrieve_episodes(user_id, query_vec, k):
    docs = list(get_episodes().find({"user_id": user_id}))
    if not docs or k <= 0:
        return []
    matrix = np.array([d["embedding"] for d in docs], dtype=np.float32)
    q = np.array(query_vec, dtype=np.float32)
    scores = matrix @ q / (np.linalg.norm(matrix, axis=1) * np.linalg.norm(q) + 1e-9)  # cosine similarity
    return [{"fact": docs[i]["fact"], "importance": docs[i]["importance"], "score": round(float(scores[i]), 4),
             "session_id": docs[i]["session_id"]} for i in np.argsort(-scores)[:k]]  # top k
```

Turning a fact into an embedding (code/api/embedder.py)

code > api > embedder.py > ...

```python
# Local sentence embeddings, unit length so cosine similarity is a dot product
import os
os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")
from .config import EMBED_MODEL
_model = None  # loaded on first use, then reused
def embed(text: str) -> list[float]:
    global _model
    if _model is None:
        from sentence_transformers import SentenceTransformer
        _model = SentenceTransformer(EMBED_MODEL)
    return [float(x) for x in _model.encode([text], normalize_embeddings=True)[0]]  # plain floats for MongoDB

```

Session summary every 5 user messages, and the lifetime summary (scope 'user', session_id empty)

```python
def summarize_session(user_id, session_id):
    last = get_summaries().find_one({"user_id": user_id, "scope": "session", "session_id": session_id},
                                    sort=[("created_at", -1), ("_id", -1)])
    query = {"user_id": user_id, "session_id": session_id}
    if last:
        query["created_at"] = {"$gt": last["created_at"]}  # only the messages since the last summary
    rows = list(get_messages().find(query).sort([("created_at", 1), ("_id", 1)]))
    text = "\n".join(f"{r['role']}: {r['content']}" for r in rows)
    prompt = f"Previous summary:\n{last['text'] if last else 'none'}\n\nNew messages:\n{text}\n\nWrite the updated summary."
    result = call_model([{"role": "system", "content": SUMMARY_SYSTEM}, {"role": "user", "content": prompt}], num_predict=250)
    doc = {"user_id": user_id, "session_id": session_id, "scope": "session", "text": result["text"].strip(), "created_at": now()}
    get_summaries().insert_one(doc)
    return result
def maybe_lifetime(user_id):
    sessions = get_summaries().find({"user_id": user_id, "scope": "session"})
    docs = list(sessions.sort([("created_at", -1), ("_id", -1)]))
    if not docs or len(docs) % config.LIFETIME_EVERY_SUMMARIES:
        return None  # only after every few session summaries
    recent = "\n\n".join(d["text"] for d in reversed(docs[:config.LIFETIME_EVERY_SUMMARIES]))
    prompt = f"Previous lifetime summary:\n{latest_summary(user_id, 'user') or 'none'}\n\nSession summaries:\n{recent}\n\nWrite the lifetime summary."
    result = call_model([{"role": "system", "content": LIFETIME_SYSTEM}, {"role": "user", "content": prompt}], num_predict=250)
    get_summaries().insert_one({"user_id": user_id, "session_id": None, "scope": "user", "text": result["text"].strip(),
                                "created_at": now()})  # session_id stays null for the lifetime summary
    return result
```

Building the prompt: primer, then summaries, then recent messages, then the new message with the facts

```python
def build_prompt(session_summary, lifetime_summary, window, message, facts):
    memory = []
    if lifetime_summary:
        memory.append("Lifetime summary:\n" + lifetime_summary)
    if session_summary:
        memory.append("Session summary:\n" + session_summary)
    long_term = "\n\n".join(memory)
    system = PRIMER + ("\n\nLong-term memory:\n" + long_term if long_term else "")
    episodic = "\n".join(f"- {f['fact']}" for f in facts)
    current = message + ("\n\nRelevant remembered facts about the user:\n" + episodic if episodic else "")
    messages = [{"role": "system", "content": system}] + window + [{"role": "user", "content": current}]  # primer, summaries, window, message + facts
    parts = {"primer": PRIMER, "long_term": long_term, "short_term": "\n".join(m["content"] for m in window),
             "current": message, "episodic": episodic}
    return messages, parts
```

3. GET /api/memory/{user_id} and GET /api/aggregate/{user_id}

The memory and aggregate endpoints (code/api/routers/chat.py)



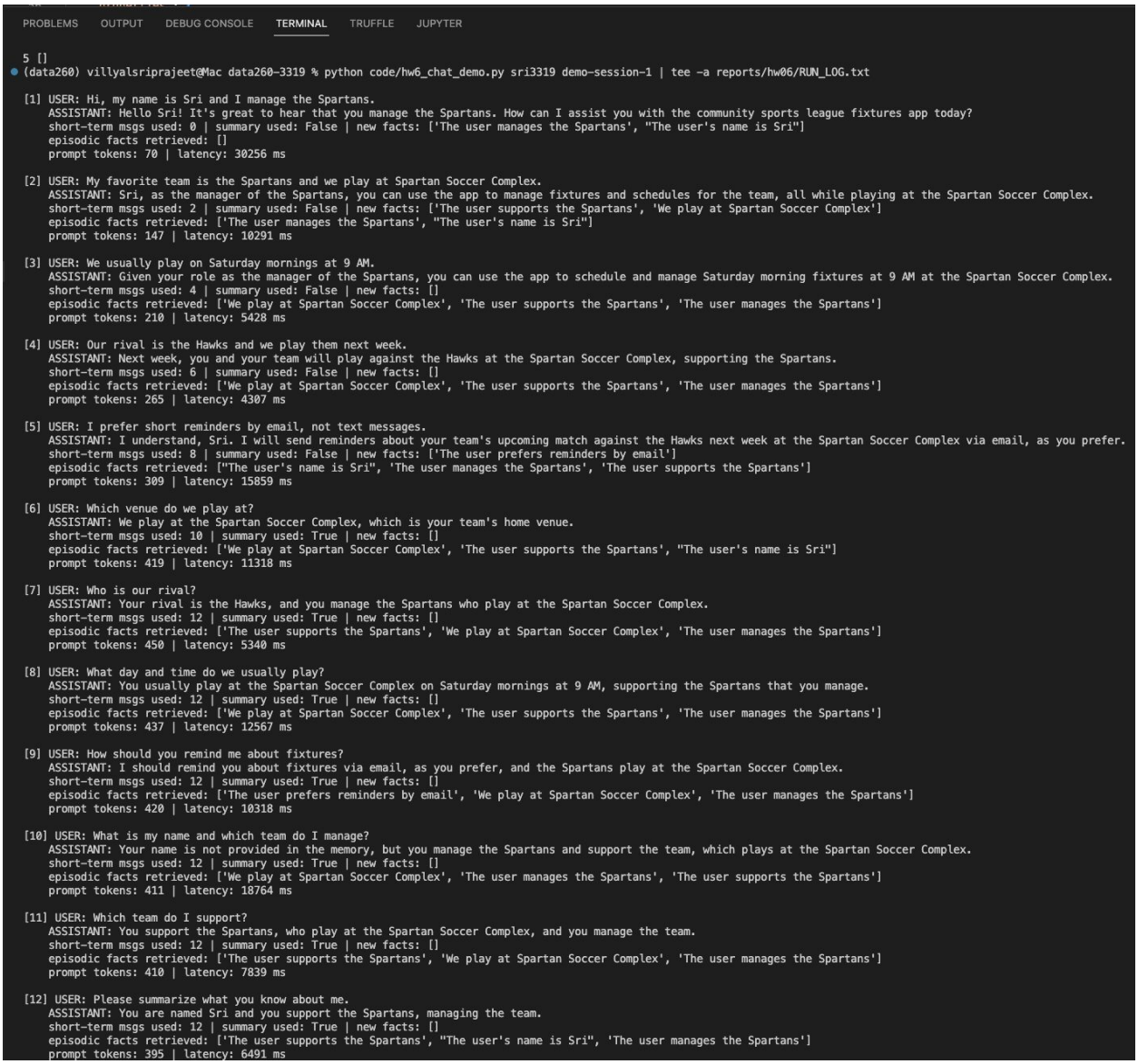
```python
@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    session_id = latest_session_id(user_id)
    rows = list(get_messages().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(16))
    facts = list(get_episodes().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(20))
    session_doc = get_summaries().find_one({"user_id": user_id, "scope": "session", "session_id": session_id},
                                           sort=[("created_at", -1), ("_id", -1)]) if session_id else None
    lifetime_doc = get_summaries().find_one({"user_id": user_id, "scope": "user"}, sort=[("created_at", -1), ("_id", -1)])
    return {"user_id": user_id, "session_id": session_id,
            "recent_messages": [clean(r) for r in reversed(rows)],  # oldest first
            "session_summary": clean(session_doc) if session_doc else None,
            "lifetime_summary": clean(lifetime_doc) if lifetime_doc else None,
            "episodic_facts": [clean(f) for f in facts]}
@router.get("/aggregate/{user_id}")
def aggregate(user_id: str):
    pipeline = [{"$match": {"user_id": user_id}},
                {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}}, "count": {"$sum": 1}}},  # messages per day
                {"$sort": {"_id": 1}}]
    daily = [{"date": d["_id"], "count": d["count"]} for d in get_messages().aggregate(pipeline)]
    recent = list(get_summaries().find({"user_id": user_id}).sort([("created_at", -1), ("_id", -1)]).limit(5))
    return {"user_id": user_id, "daily_message_counts": daily, "recent_summaries": [clean(s) for s in recent]}

```

4. Conversation and results

I sent 12 messages in one session (user sri3319) using code/hw6_chat_demo.py. It calls /api/chat and prints what memory was used on each turn. You can see the short-term window grow to 12 messages, the session summary appear after the 5th user message, and saved facts being pulled in from turn 2 onwards.

The conversation - each reply and the memory it used



On turn 10 the model said it didn't know my name, even though "The user's name is Sri" was saved. Only the top 3 facts get pulled in, and that turn three other facts ranked higher, so the name never reached the prompt. By turn 12 it used the name again. Probe P1 in Part 3 shows the same limit.

GET /api/memory/sri3319 - the last 16 messages (first part)

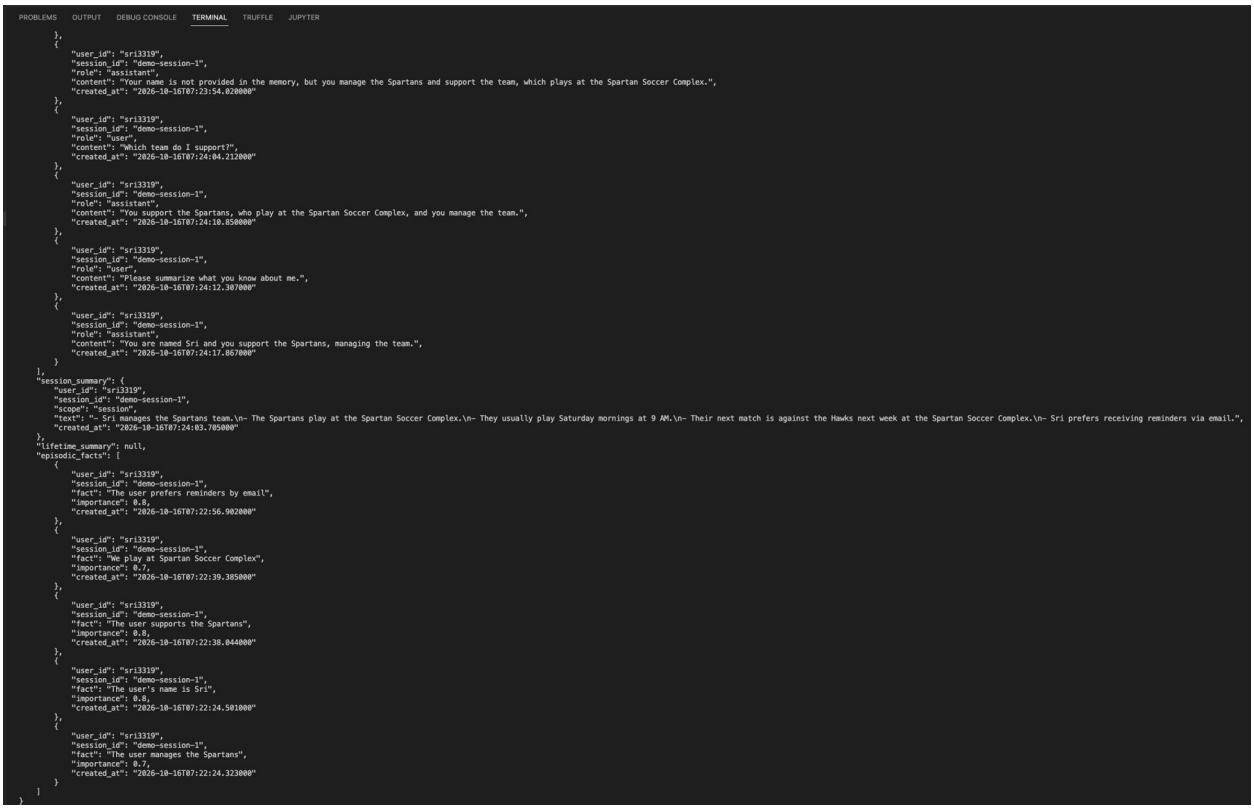


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

    prompt tokens: 395 | latency: 6491 ms
(data260) villyalsriprajeet@Mac data260-3319 % curl -s http://127.0.0.1:8619/api/memory/sri3319 | python -m json.tool | tee -a reports/hw06/RUN_LOG.txt

{
    "user_id": "sri3319",
    "session_id": "demo-session-1",
    "recent_messages": [
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "I prefer short reminders by email, not text messages.",
            "created_at": "2026-10-16T07:22:49.464000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "I understand, Sri. I will send reminders about your team's upcoming match against the Hawks next week at the Spartan Soccer Complex via email, as you prefer.",
            "created_at": "2026-10-16T07:22:54.163000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Which venue do we play at?",
            "created_at": "2026-10-16T07:23:05.428000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "We play at the Spartan Soccer Complex, which is your team's home venue.",
            "created_at": "2026-10-16T07:23:14.118000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "Who is our rival?",
            "created_at": "2026-10-16T07:23:16.832000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "Your rival is the Hawks, and you manage the Spartans who play at the Spartan Soccer Complex.",
            "created_at": "2026-10-16T07:23:20.975000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "What day and time do we usually play?",
            "created_at": "2026-10-16T07:23:22.243000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "You usually play at the Spartan Soccer Complex on Saturday mornings at 9 AM, supporting the Spartans that you manage.",
            "created_at": "2026-10-16T07:23:33.619000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "How should you remind me about fixtures?",
            "created_at": "2026-10-16T07:23:34.933000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "assistant",
            "content": "I should remind you about fixtures via email, as you prefer, and the Spartans play at the Spartan Soccer Complex.",
            "created_at": "2026-10-16T07:23:44.051000"
        },
        {
            "user_id": "sri3319",
            "session_id": "demo-session-1",
            "role": "user",
            "content": "What is my name and which team do I manage?",
            "created_at": "2026-10-16T07:23:45.355000"
        },
        {
            "user_id": "sri3319",
```

GET /api/memory/sri3319 - session summary, lifetime summary and recent facts

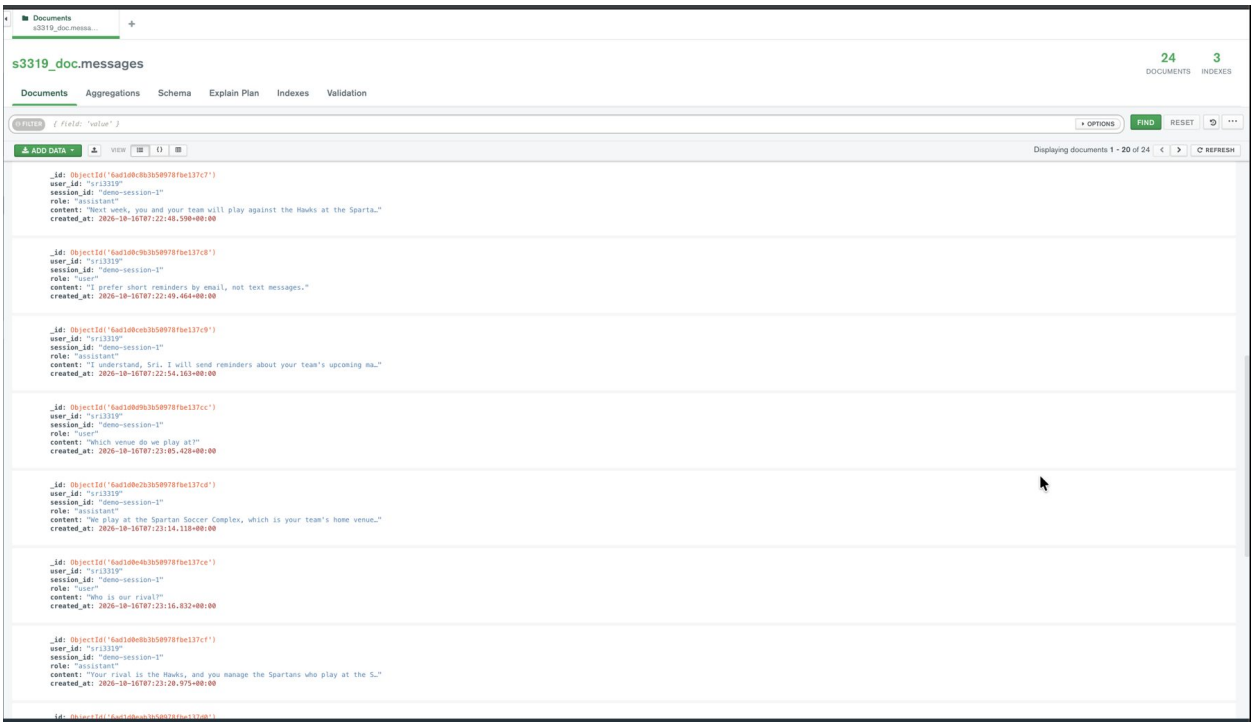


The lifetime summary is empty for this user because there are only 2 session summaries so far, and it is built after every 3. The users from the Part 3 sweep had 6 session summaries each, and MongoDB holds 8 lifetime summaries for them.

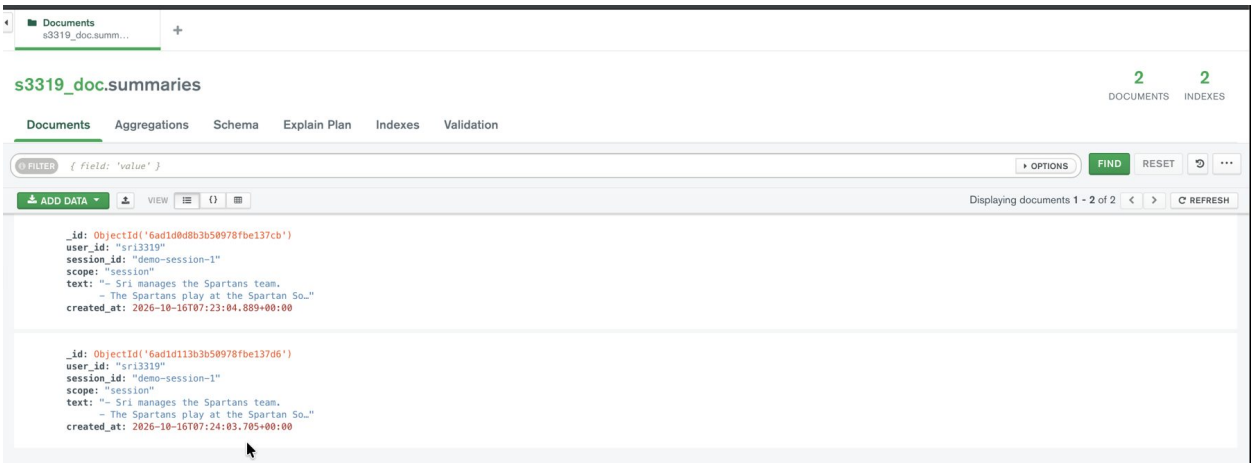
GET /api/aggregate/sri3319 - messages per day and the latest summaries



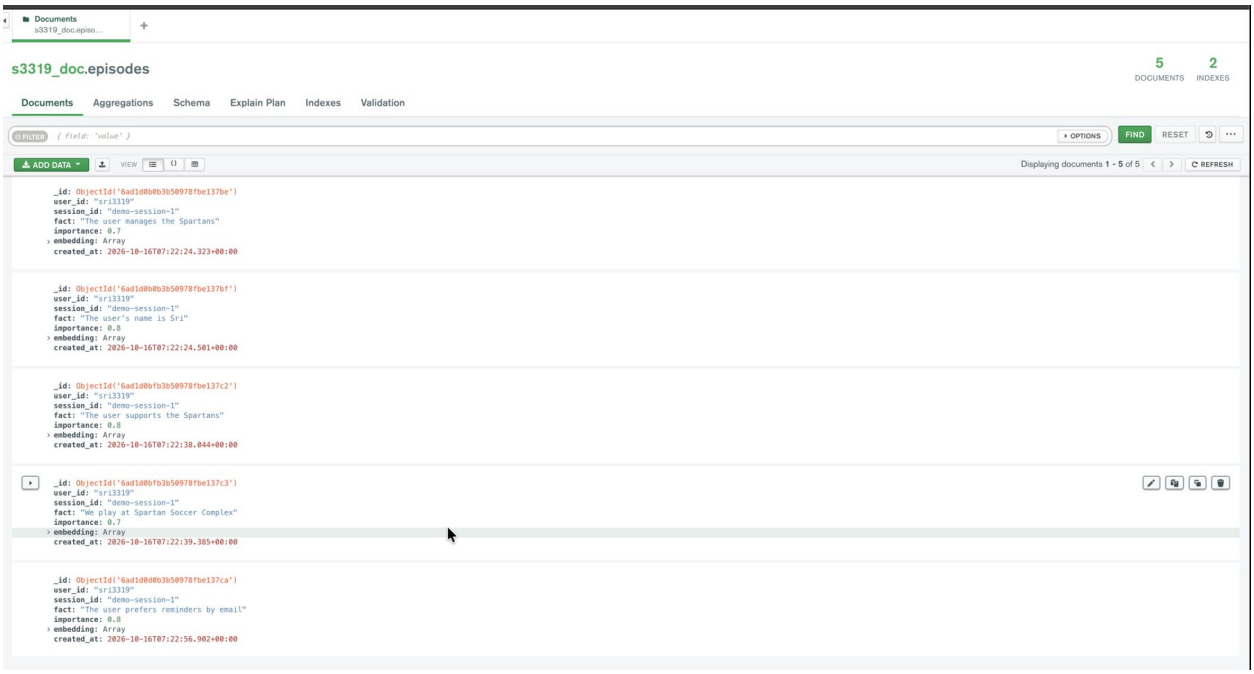
MongoDB Compass - messages collection (24 documents)



MongoDB Compass - summaries collection



MongoDB Compass - episodes collection with the fact, importance and embedding



Part 3 - What Memory Costs

1. Log memory usage

Every /api/chat request adds one row to a CSV file. Ollama gives me the prompt and reply token counts. I count the tokens for each part of the prompt myself with the Qwen2.5 tokenizer: the primer, the summaries, the recent messages, the new message and the facts. I also log the extra model calls that keep the memory up to date (fact extraction and summaries) and how long the whole request took.

The logged fields and token counting (code/api/memory.py)

```python
METRIC_FIELDS = ["timestamp", "user_id", "session_id", "turn", "short_term_n", "prompt_tokens", "completion_tokens",
                 "primer_tokens", "long_term_tokens", "short_term_tokens", "current_tokens", "episodic_tokens",
                 "extract_prompt_tokens", "extract_completion_tokens", "summary_prompt_tokens", "summary_completion_tokens",
                 "latency_ms", "reply_latency_ms", "token_method"]
_tokenizer = None
def now():
    return datetime.now(timezone.utc)
def call_model(messages, json_mode=False, num_predict=None):
    client = ModelClient(model=config.LLM_MODEL, base_url=config.OLLAMA_URL, temperature=0.0, num_ctx=config.LLM_NUM_CTX,
                         num_predict=num_predict or config.LLM_NUM_PREDICT, seed=config.SEED)  # temperature 0 and a fixed seed
    return client.complete(messages, json_mode=json_mode)
def token_method():
    return "qwen2.5 tokenizer" if _tokenizer else "chars/4 estimate"
def count_tokens(text):
    global _tokenizer
    if not text:
        return 0
    if _tokenizer is None:
        try:
            from transformers import AutoTokenizer
            _tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-3B-Instruct")  # same tokenizer family as qwen2.5:3b
        except Exception:
            _tokenizer = False  # not available, fall back to about 4 characters per token
    return len(_tokenizer.encode(text)) if _tokenizer else max(1, len(text) // 4)
```

Writing a CSV row, and the eviction policy used in step 4 (code/api/memory.py)

```python
def log_metrics(metrics, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    new_file = not path.exists()
    with open(path, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=METRIC_FIELDS, extrasaction="ignore")
        if new_file:
            writer.writeheader()
        writer.writerow(metrics)
def compress_window(window, keep_full=4):
    old, recent = window[:-keep_full], window[-keep_full:]
    return [m for m in old if m["role"] == "user"] + recent  # evict old assistant replies, keep the user's own words
```

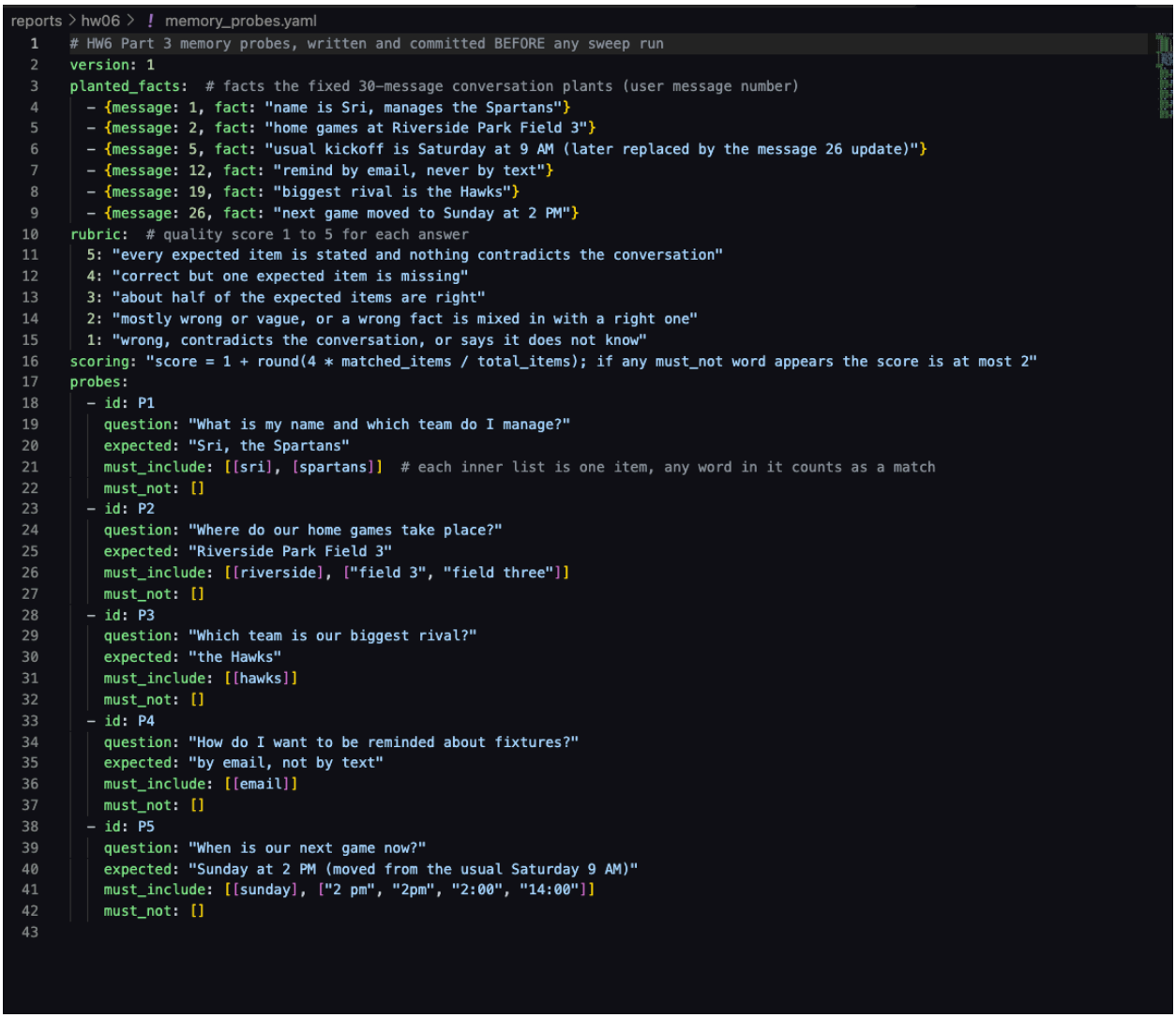
The first rows of raw/sweep_turns.csv

```
(data260) villyalsriprajeet@Mac data260-3319 % head -8 reports/hw06/raw/sweep_turns.csv | cut -d, -f2,4-12,17 | column -s, -t
user_id   turn  short_term_n  prompt_tokens  completion_tokens  primer_tokens  long_term_tokens  short_term_tokens  current_tokens  episodic_tokens  latency_ms
sweep-n4  1     4            70             30                 44             0                 0                  13              0                11882
sweep-n4  2     4            143            16                 44             0                 42                 11              15               7402
sweep-n4  3     4            191            15                 44             0                 68                 11              27               3843
sweep-n4  4     4            175            21                 44             0                 51                 12              27               4374
sweep-n4  5     4            180            29                 44             0                 57                 11              27               16544
sweep-n4  6     4            264            26                 44             63                71                 9               32               6697
sweep-n4  7     4            269            57                 44             63                73                 12              32               9694
(data260) villyalsriprajeet@Mac data260-3319 %
```

2. Memory probes

I wrote memory_probes.yaml and committed it (83f7a8c) before running anything. It has five questions about facts from the conversation, the answer I expect for each, and a simple score from 1 to 5 based on how many of the expected words show up in the answer.

reports/hw06/memory_probes.yaml



```
reports > hw06 > ! memory_probes.yaml
# HW6 Part 3 memory probes, written and committed BEFORE any sweep run
version: 1
planted_facts:  # facts the fixed 30-message conversation plants (user message number)
  - {message: 1, fact: "name is Sri, manages the Spartans"}
  - {message: 2, fact: "home games at Riverside Park Field 3"}
  - {message: 5, fact: "usual kickoff is Saturday at 9 AM (later replaced by the message 26 update)"}
  - {message: 12, fact: "remind by email, never by text"}
  - {message: 19, fact: "biggest rival is the Hawks"}
  - {message: 26, fact: "next game moved to Sunday at 2 PM"}
rubric:  # quality score 1 to 5 for each answer
  5: "every expected item is stated and nothing contradicts the conversation"
  4: "correct but one expected item is missing"
  3: "about half of the expected items are right"
  2: "mostly wrong or vague, or a wrong fact is mixed in with a right one"
  1: "wrong, contradicts the conversation, or says it does not know"
scoring: "score = 1 + round(4 * matched_items / total_items); if any must_not word appears the score is at most 2"
probes:
  - id: P1
    question: "What is my name and which team do I manage?"
    expected: "Sri, the Spartans"
    must_include: [[sri], [spartans]]  # each inner list is one item, any word in it counts as a match
    must_not: []
  - id: P2
    question: "Where do our home games take place?"
    expected: "Riverside Park Field 3"
    must_include: [[riverside], ["field 3", "field three"]]
    must_not: []
  - id: P3
    question: "Which team is our biggest rival?"
    expected: "the Hawks"
    must_include: [[hawks]]
    must_not: []
  - id: P4
    question: "How do I want to be reminded about fixtures?"
    expected: "by email, not by text"
    must_include: [[email]]
    must_not: []
  - id: P5
    question: "When is our next game now?"
    expected: "Sunday at 2 PM (moved from the usual Saturday 9 AM)"
    must_include: [[sunday], ["2 pm", "2pm", "2:00", "14:00"]]
    must_not: []
```

3. Compare short-term memory sizes

code/hw6_sweep.py sends the same 30 messages for every setting, each time as a fresh user. Six of the messages (1, 2, 5, 12, 19 and 26) contain facts, and the rest are ordinary league questions. Message 26 moves the next game to Sunday at 2 PM. After message 30 it asks the five probes, and nothing from the probes is saved. The model is warmed up first so the first turn isn't slower than the rest.

The settings, the planted facts and how answers are scored (code/hw6_sweep.py)

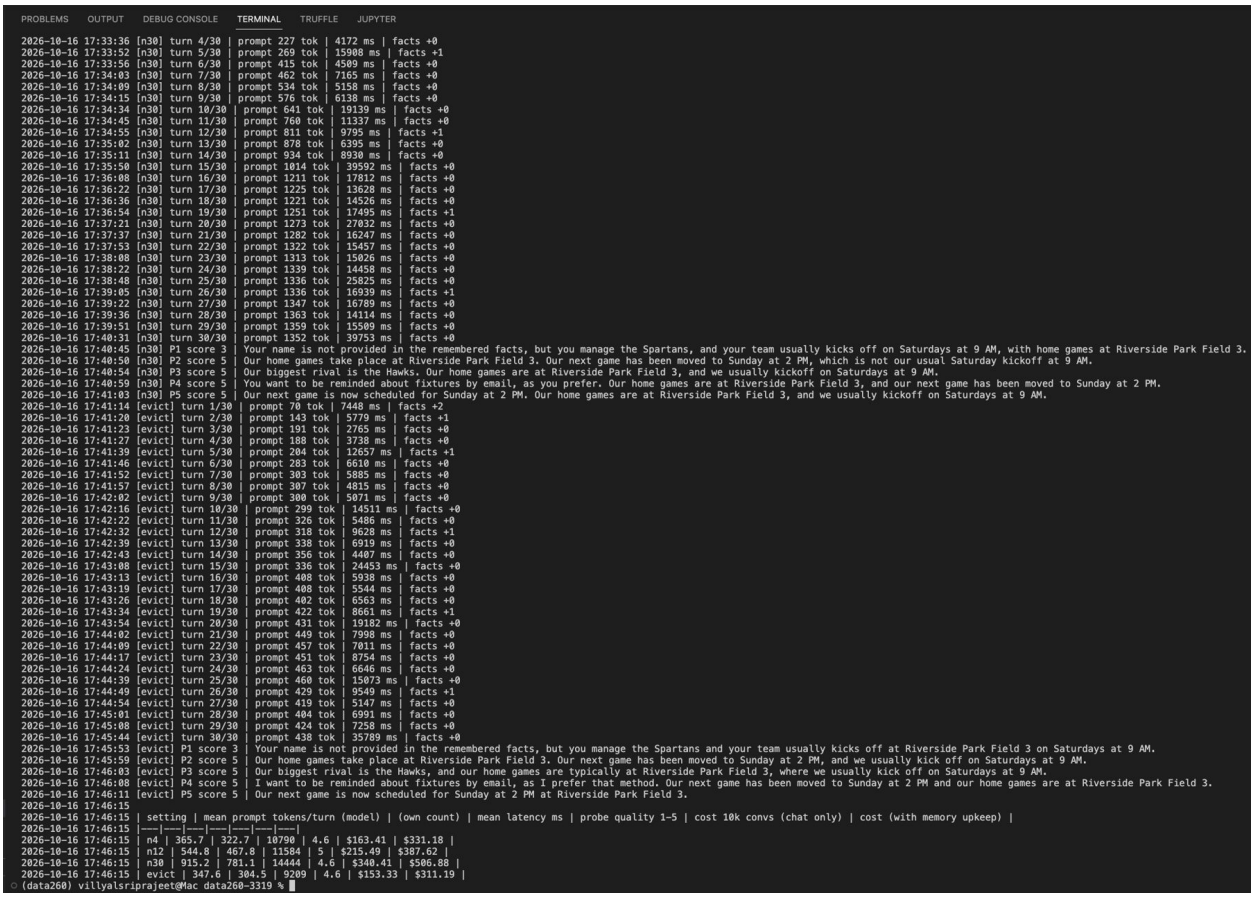
```python
ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "reports" / "hw06" / "raw"
TURNS_CSV = RAW / "sweep_turns.csv"
PROBES = yaml.safe_load(open(ROOT / "reports" / "hw06" / "memory_probes.yaml"))["probes"]
SETTINGS = {"n4": (4, None), "n12": (12, None), "n30": (30, None), "evict": (12, "evict_assistant")}  # label: (window size, policy)
FILLER = ["How many points does a win give in most leagues?", "What does the Postponed status mean for a fixture?",
          "Can a fixture be both Live and Completed?", "What is the difference between a League and a Cup fixture?",
          "How long is a typical youth soccer match?", "What should I check before marking a fixture as Completed?",
          "How do tie-breakers usually work in a league table?", "Is a Friendly counted in the standings?",
          "What is a good way to name tournament rounds?", "How many substitutes are usually allowed?",
          "What does extra time mean in a cup match?", "Why might a fixture be postponed?",
          "How early should teams arrive before kickoff?", "What is a walkover?",
          "How are home and away fixtures usually decided?", "What is a bye in a tournament?",
          "How should results be recorded after a match?", "What is goal difference?",
          "How do I report a rule dispute?", "What does a clean sheet mean?",
          "How many teams usually play in a small community league?", "What is a rescheduled fixture?",
          "How long before a match can a fixture be postponed?", "Thanks, that is all for today."]
PLANTED = {1: "Hi, my name is Sri and I manage the Spartans.", 2: "Our home games are at Riverside Park Field 3.",
           5: "We usually kick off on Saturdays at 9 AM.", 12: "Please remind me about fixtures by email, never by text.",
           19: "Our biggest rival is the Hawks.", 26: "Update: our next game has been moved to Sunday at 2 PM."}
def conversation():
    filler = iter(FILLER)
    return [PLANTED.get(i) or next(filler) for i in range(1, 31)]  # 30 user messages, facts planted at fixed positions
def score(answer, probe):
    text = answer.lower()
    hit = lambda w: re.search(r"\b" + re.escape(str(w).lower()) + r"\b", text) is not None
    groups = probe["must_include"]
    matched = sum(any(hit(w) for w in g) for g in groups)
    value = 1 if matched == 0 else 1 + round(4 * matched / len(groups))
    if any(hit(w) for w in probe.get("must_not") or []):
        value = min(value, 2)
    return value, matched
```

Start of the sweep in RUN_LOG.txt - tokens and time per turn for N=4 and N=12

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

(data260) villyalsriprajeet@Mac data260-3319 % caffeinate -i python -u -m code.hw6_sweep 2>&1 | while IFS= read -r l; do echo "$(date '+%Y-%m-%d %H:%M:%S') $l"; done | tee -a reports/hw06/RUN_LOG.txt
2026-10-16 17:21:03 /Users/villyalsriprajeet/miniconda3_x86/envs/data260/lib/python3.12/site-packages/huggingface_hub/file_download.py:1132: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
2026-10-16 17:21:03  warnings.warn(
2026-10-16 17:21:14 Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
2026-10-16 17:21:26 [n4] turn 1/30 | prompt 70 tok | 11882 ms | facts +2
2026-10-16 17:21:33 [n4] turn 2/30 | prompt 143 tok | 7402 ms | facts +1
2026-10-16 17:21:37 [n4] turn 3/30 | prompt 191 tok | 3843 ms | facts +0
2026-10-16 17:21:41 [n4] turn 4/30 | prompt 175 tok | 4374 ms | facts +0
2026-10-16 17:21:58 [n4] turn 5/30 | prompt 180 tok | 16544 ms | facts +1
2026-10-16 17:22:05 [n4] turn 6/30 | prompt 264 tok | 6697 ms | facts +0
2026-10-16 17:22:14 [n4] turn 7/30 | prompt 269 tok | 9694 ms | facts +0
2026-10-16 17:22:21 [n4] turn 8/30 | prompt 295 tok | 6202 ms | facts +0
2026-10-16 17:22:27 [n4] turn 9/30 | prompt 296 tok | 6521 ms | facts +0
2026-10-16 17:22:49 [n4] turn 10/30 | prompt 270 tok | 21519 ms | facts +0
2026-10-16 17:22:56 [n4] turn 11/30 | prompt 351 tok | 7567 ms | facts +0
2026-10-16 17:23:08 [n4] turn 12/30 | prompt 342 tok | 11600 ms | facts +1
2026-10-16 17:23:14 [n4] turn 13/30 | prompt 343 tok | 6189 ms | facts +0
2026-10-16 17:23:20 [n4] turn 14/30 | prompt 365 tok | 5416 ms | facts +0
2026-10-16 17:23:59 [n4] turn 15/30 | prompt 351 tok | 39826 ms | facts +0
2026-10-16 17:24:08 [n4] turn 16/30 | prompt 473 tok | 8705 ms | facts +0
2026-10-16 17:24:14 [n4] turn 17/30 | prompt 486 tok | 5578 ms | facts +0
2026-10-16 17:24:24 [n4] turn 18/30 | prompt 473 tok | 9734 ms | facts +0
2026-10-16 17:24:33 [n4] turn 19/30 | prompt 508 tok | 9132 ms | facts +1
2026-10-16 17:24:53 [n4] turn 20/30 | prompt 522 tok | 19781 ms | facts +0
2026-10-16 17:25:02 [n4] turn 21/30 | prompt 474 tok | 9318 ms | facts +0
2026-10-16 17:25:09 [n4] turn 22/30 | prompt 499 tok | 7177 ms | facts +0
2026-10-16 17:25:17 [n4] turn 23/30 | prompt 473 tok | 7419 ms | facts +0
2026-10-16 17:25:22 [n4] turn 24/30 | prompt 457 tok | 5216 ms | facts +0
2026-10-16 17:25:38 [n4] turn 25/30 | prompt 443 tok | 16558 ms | facts +0
2026-10-16 17:25:49 [n4] turn 26/30 | prompt 459 tok | 10249 ms | facts +1
2026-10-16 17:25:54 [n4] turn 27/30 | prompt 461 tok | 5163 ms | facts +0
2026-10-16 17:25:59 [n4] turn 28/30 | prompt 454 tok | 5086 ms | facts +0
2026-10-16 17:26:04 [n4] turn 29/30 | prompt 448 tok | 5181 ms | facts +0
2026-10-16 17:26:40 [n4] turn 30/30 | prompt 437 tok | 34124 ms | facts +0
2026-10-16 17:26:47 [n4] P1 score 3 | Your name is not provided in the remembered facts, but you manage the Spartans and your home games are at Riverside Park Field 3.
2026-10-16 17:26:49 [n4] P2 score 5 | Our home games take place at Riverside Park Field 3.
2026-10-16 17:26:52 [n4] P3 score 5 | Our biggest rival is the Hawks, and our home games are still scheduled at Riverside Park Field 3.
2026-10-16 17:26:56 [n4] P4 score 5 | I want to be reminded about fixtures by email, with our home games at Riverside Park Field 3 and our next game scheduled for Sunday at 2 PM.
2026-10-16 17:26:59 [n4] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
2026-10-16 17:27:08 [n12] turn 1/30 | prompt 70 tok | 7882 ms | facts +2
2026-10-16 17:27:13 [n12] turn 2/30 | prompt 143 tok | 5437 ms | facts +1
2026-10-16 17:27:16 [n12] turn 3/30 | prompt 191 tok | 2855 ms | facts +0
2026-10-16 17:27:20 [n12] turn 4/30 | prompt 227 tok | 4147 ms | facts +0
2026-10-16 17:27:38 [n12] turn 5/30 | prompt 269 tok | 17401 ms | facts +1
2026-10-16 17:27:45 [n12] turn 6/30 | prompt 415 tok | 7765 ms | facts +0
2026-10-16 17:27:54 [n12] turn 7/30 | prompt 462 tok | 8892 ms | facts +0
2026-10-16 17:28:02 [n12] turn 8/30 | prompt 482 tok | 7788 ms | facts +0
2026-10-16 17:28:13 [n12] turn 9/30 | prompt 488 tok | 10550 ms | facts +0
2026-10-16 17:28:38 [n12] turn 10/30 | prompt 514 tok | 25480 ms | facts +0
2026-10-16 17:28:46 [n12] turn 11/30 | prompt 585 tok | 7588 ms | facts +0
2026-10-16 17:28:58 [n12] turn 12/30 | prompt 557 tok | 12068 ms | facts +1
2026-10-16 17:29:08 [n12] turn 13/30 | prompt 585 tok | 9724 ms | facts +0
2026-10-16 17:29:15 [n12] turn 14/30 | prompt 566 tok | 7142 ms | facts +0
2026-10-16 17:29:51 [n12] turn 15/30 | prompt 569 tok | 36089 ms | facts +0
2026-10-16 17:30:00 [n12] turn 16/30 | prompt 687 tok | 8812 ms | facts +0
2026-10-16 17:30:07 [n12] turn 17/30 | prompt 663 tok | 7186 ms | facts +0
2026-10-16 17:30:16 [n12] turn 18/30 | prompt 662 tok | 8651 ms | facts +0
2026-10-16 17:30:26 [n12] turn 19/30 | prompt 658 tok | 10166 ms | facts +1
2026-10-16 17:30:46 [n12] turn 20/30 | prompt 653 tok | 19925 ms | facts +0
2026-10-16 17:30:54 [n12] turn 21/30 | prompt 700 tok | 8264 ms | facts +0
2026-10-16 17:31:02 [n12] turn 22/30 | prompt 707 tok | 7614 ms | facts +0
2026-10-16 17:31:08 [n12] turn 23/30 | prompt 708 tok | 6497 ms | facts +0
2026-10-16 17:31:18 [n12] turn 24/30 | prompt 696 tok | 9679 ms | facts +0
2026-10-16 17:31:35 [n12] turn 25/30 | prompt 700 tok | 16905 ms | facts +0
2026-10-16 17:31:46 [n12] turn 26/30 | prompt 672 tok | 11038 ms | facts +1
2026-10-16 17:31:53 [n12] turn 27/30 | prompt 672 tok | 6939 ms | facts +0
2026-10-16 17:32:01 [n12] turn 28/30 | prompt 675 tok | 7780 ms | facts +0
2026-10-16 17:32:08 [n12] turn 29/30 | prompt 678 tok | 7525 ms | facts +0
2026-10-16 17:32:48 [n12] turn 30/30 | prompt 691 tok | 39729 ms | facts +0
2026-10-16 17:32:57 [n12] P1 score 5 | Your name is Sri and you manage the Spartans, who usually kick off home games at Riverside Park Field 3 on Saturdays at 9 AM.
2026-10-16 17:33:01 [n12] P2 score 5 | Our home games take place at Riverside Park Field 3, as usual. However, our next game has been moved to Sunday at 2 PM due to a postponement.
2026-10-16 17:33:05 [n12] P3 score 5 | Our biggest rival is the Hawks, and we usually play our home games at Riverside Park Field 3 on Saturdays at 9 AM.
2026-10-16 17:33:09 [n12] P4 score 5 | I want to be reminded about fixtures by email, as you prefer, and our next game has been moved to Sunday at 2 PM.
2026-10-16 17:33:12 [n12] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM.
```

End of the sweep - N=30 and eviction, the probe scores and the summary table



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TRUFFLE  JUPYTER

2026-10-16 17:33:36 [n30] turn 4/30 | prompt 227 tok | 4172 ms | facts +0
2026-10-16 17:33:52 [n30] turn 5/30 | prompt 269 tok | 15908 ms | facts +1
2026-10-16 17:33:56 [n30] turn 6/30 | prompt 415 tok | 4509 ms | facts +0
2026-10-16 17:34:03 [n30] turn 7/30 | prompt 462 tok | 7165 ms | facts +0
2026-10-16 17:34:09 [n30] turn 8/30 | prompt 534 tok | 5158 ms | facts +0
2026-10-16 17:34:15 [n30] turn 9/30 | prompt 576 tok | 6138 ms | facts +0
2026-10-16 17:34:34 [n30] turn 10/30 | prompt 641 tok | 19139 ms | facts +0
2026-10-16 17:34:45 [n30] turn 11/30 | prompt 760 tok | 11337 ms | facts +0
2026-10-16 17:34:55 [n30] turn 12/30 | prompt 811 tok | 9795 ms | facts +1
2026-10-16 17:35:02 [n30] turn 13/30 | prompt 878 tok | 6395 ms | facts +0
2026-10-16 17:35:11 [n30] turn 14/30 | prompt 934 tok | 8930 ms | facts +0
2026-10-16 17:35:50 [n30] turn 15/30 | prompt 1014 tok | 39592 ms | facts +0
2026-10-16 17:36:08 [n30] turn 16/30 | prompt 1211 tok | 17812 ms | facts +0
2026-10-16 17:36:22 [n30] turn 17/30 | prompt 1225 tok | 13628 ms | facts +0
2026-10-16 17:36:36 [n30] turn 18/30 | prompt 1221 tok | 14526 ms | facts +0
2026-10-16 17:36:54 [n30] turn 19/30 | prompt 1251 tok | 17495 ms | facts +1
2026-10-16 17:37:21 [n30] turn 20/30 | prompt 1273 tok | 27032 ms | facts +0
2026-10-16 17:37:37 [n30] turn 21/30 | prompt 1282 tok | 16247 ms | facts +0
2026-10-16 17:37:53 [n30] turn 22/30 | prompt 1322 tok | 15457 ms | facts +0
2026-10-16 17:38:08 [n30] turn 23/30 | prompt 1313 tok | 15026 ms | facts +0
2026-10-16 17:38:22 [n30] turn 24/30 | prompt 1339 tok | 14458 ms | facts +0
2026-10-16 17:38:48 [n30] turn 25/30 | prompt 1336 tok | 25825 ms | facts +0
2026-10-16 17:39:05 [n30] turn 26/30 | prompt 1336 tok | 16939 ms | facts +1
2026-10-16 17:39:22 [n30] turn 27/30 | prompt 1347 tok | 16789 ms | facts +0
2026-10-16 17:39:36 [n30] turn 28/30 | prompt 1363 tok | 14114 ms | facts +0
2026-10-16 17:39:51 [n30] turn 29/30 | prompt 1359 tok | 15509 ms | facts +0
2026-10-16 17:40:31 [n30] turn 30/30 | prompt 1352 tok | 39753 ms | facts +0
2026-10-16 17:40:45 [n30] P1 score 3 | Your name is not provided in the remembered facts, but you manage the Spartans, and your team usually kicks off on Saturdays at 9 AM, with home games at Riverside Park Field 3.
2026-10-16 17:40:50 [n30] P2 score 5 | Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunday at 2 PM, which is not our usual Saturday kickoff at 9 AM.
2026-10-16 17:40:54 [n30] P3 score 5 | Our biggest rival is the Hawks. Our home games are at Riverside Park Field 3, and we usually kickoff on Saturdays at 9 AM.
2026-10-16 17:40:59 [n30] P4 score 5 | You want to be reminded about fixtures by email, as you prefer. Our home games are at Riverside Park Field 3, and our next game has been moved to Sunday at 2 PM.
2026-10-16 17:41:03 [n30] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM. Our home games are at Riverside Park Field 3, and we usually kickoff on Saturdays at 9 AM.
2026-10-16 17:41:14 [evict] turn 1/30 | prompt 70 tok | 7448 ms | facts +2
2026-10-16 17:41:20 [evict] turn 2/30 | prompt 143 tok | 5779 ms | facts +1
2026-10-16 17:41:23 [evict] turn 3/30 | prompt 191 tok | 2765 ms | facts +0
2026-10-16 17:41:27 [evict] turn 4/30 | prompt 188 tok | 3738 ms | facts +0
2026-10-16 17:41:39 [evict] turn 5/30 | prompt 204 tok | 12657 ms | facts +1
2026-10-16 17:41:46 [evict] turn 6/30 | prompt 283 tok | 6610 ms | facts +0
2026-10-16 17:41:52 [evict] turn 7/30 | prompt 303 tok | 5885 ms | facts +0
2026-10-16 17:41:57 [evict] turn 8/30 | prompt 307 tok | 4815 ms | facts +0
2026-10-16 17:42:02 [evict] turn 9/30 | prompt 300 tok | 5071 ms | facts +0
2026-10-16 17:42:16 [evict] turn 10/30 | prompt 299 tok | 14511 ms | facts +0
2026-10-16 17:42:22 [evict] turn 11/30 | prompt 326 tok | 5486 ms | facts +0
2026-10-16 17:42:32 [evict] turn 12/30 | prompt 318 tok | 9628 ms | facts +1
2026-10-16 17:42:39 [evict] turn 13/30 | prompt 338 tok | 6919 ms | facts +0
2026-10-16 17:42:43 [evict] turn 14/30 | prompt 356 tok | 4407 ms | facts +0
2026-10-16 17:43:08 [evict] turn 15/30 | prompt 336 tok | 24453 ms | facts +0
2026-10-16 17:43:13 [evict] turn 16/30 | prompt 408 tok | 5938 ms | facts +0
2026-10-16 17:43:19 [evict] turn 17/30 | prompt 408 tok | 5544 ms | facts +0
2026-10-16 17:43:26 [evict] turn 18/30 | prompt 402 tok | 6563 ms | facts +0
2026-10-16 17:43:34 [evict] turn 19/30 | prompt 422 tok | 8661 ms | facts +1
2026-10-16 17:43:54 [evict] turn 20/30 | prompt 431 tok | 19182 ms | facts +0
2026-10-16 17:44:02 [evict] turn 21/30 | prompt 449 tok | 7998 ms | facts +0
2026-10-16 17:44:09 [evict] turn 22/30 | prompt 457 tok | 7011 ms | facts +0
2026-10-16 17:44:17 [evict] turn 23/30 | prompt 451 tok | 8754 ms | facts +0
2026-10-16 17:44:24 [evict] turn 24/30 | prompt 463 tok | 6646 ms | facts +0
2026-10-16 17:44:39 [evict] turn 25/30 | prompt 460 tok | 15073 ms | facts +0
2026-10-16 17:44:49 [evict] turn 26/30 | prompt 429 tok | 9549 ms | facts +1
2026-10-16 17:44:54 [evict] turn 27/30 | prompt 419 tok | 5147 ms | facts +0
2026-10-16 17:45:01 [evict] turn 28/30 | prompt 404 tok | 6991 ms | facts +0
2026-10-16 17:45:08 [evict] turn 29/30 | prompt 424 tok | 7258 ms | facts +0
2026-10-16 17:45:44 [evict] turn 30/30 | prompt 438 tok | 35789 ms | facts +0
2026-10-16 17:45:53 [evict] P1 score 3 | Your name is not provided in the remembered facts, but you manage the Spartans and your team usually kicks off at Riverside Park Field 3 on Saturdays at 9 AM.
2026-10-16 17:45:59 [evict] P2 score 5 | Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunday at 2 PM, and we usually kick off on Saturdays at 9 AM.
2026-10-16 17:46:03 [evict] P3 score 5 | Our biggest rival is the Hawks, and our home games are typically at Riverside Park Field 3, where we usually kick off on Saturdays at 9 AM.
2026-10-16 17:46:08 [evict] P4 score 5 | I want to be reminded about fixtures by email, as I prefer that method. Our next game has been moved to Sunday at 2 PM and our home games are at Riverside Park Field 3.
2026-10-16 17:46:11 [evict] P5 score 5 | Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
2026-10-16 17:46:15
2026-10-16 17:46:15 | setting | mean prompt tokens/turn (model) | (own count) | mean latency ms | probe quality 1–5 | cost 10k convs (chat only) | cost (with memory upkeep) |
2026-10-16 17:46:15 |---|---|---|---|---|---|---|
2026-10-16 17:46:15 | n4 | 365.7 | 322.7 | 10790 | 4.6 | $163.41 | $331.18 |
2026-10-16 17:46:15 | n12 | 544.8 | 467.8 | 11584 | 5 | $215.49 | $387.62 |
2026-10-16 17:46:15 | n30 | 915.2 | 781.1 | 14444 | 4.6 | $340.41 | $506.88 |
2026-10-16 17:46:15 | evict | 347.6 | 304.5 | 9209 | 4.6 | $153.33 | $311.19 |
(data260) villyalsriprajeet@Mac data260-3319 %
```

Each probe answer and its score, per setting

```
(data260) villyalsriprajeet@Mac data260-3319 % python - <<'E'
import json
r = json.load(open('reports/hw06/raw/probe_results.json'))
for k, v in r.items():
    print(k, v['mean_probe_quality'])
    for p in v['probes']: print('  ', p['id'], p['score'], p['answer'][:90])
E
n4 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans and your ho
   P2 5 Our home games take place at Riverside Park Field 3.
   P3 5 Our biggest rival is the Hawks, and our home games are still scheduled at Riverside Park F
   P4 5 I want to be reminded about fixtures by email, with our home games at Riverside Park Field
   P5 5 Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
n12 5
   P1 5 Your name is Sri and you manage the Spartans, who usually kick off home games at Riverside
   P2 5 Our home games take place at Riverside Park Field 3, as usual. However, our next game has
   P3 5 Our biggest rival is the Hawks, and we usually play our home games at Riverside Park Field
   P4 5 I want to be reminded about fixtures by email, as you prefer, and our next game has been m
   P5 5 Our next game is now scheduled for Sunday at 2 PM.
n30 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans, and your t
   P2 5 Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunda
   P3 5 Our biggest rival is the Hawks. Our home games are at Riverside Park Field 3, and we usual
   P4 5 You want to be reminded about fixtures by email, as you prefer. Our home games are at Rive
   P5 5 Our next game is now scheduled for Sunday at 2 PM. Our home games are at Riverside Park Fi
evict 4.6
   P1 3 Your name is not provided in the remembered facts, but you manage the Spartans and your te
   P2 5 Our home games take place at Riverside Park Field 3. Our next game has been moved to Sunda
   P3 5 Our biggest rival is the Hawks, and our home games are typically at Riverside Park Field 3
   P4 5 I want to be reminded about fixtures by email, as I prefer that method. Our next game has
   P5 5 Our next game is now scheduled for Sunday at 2 PM at Riverside Park Field 3.
(data260) villyalsriprajeet@Mac data260-3319 %
```

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	365.7	10,790	4.6
SHORT_TERM_N=12	544.8	11,584	5.0
SHORT_TERM_N=30	915.2	14,444	4.6
Eviction/compression policy (N=12, evict_assistant)	347.6	9,209	4.6

The bigger the window, the bigger the prompt. N=30 sends about 2.5 times the tokens of N=4 and is the slowest. All settings except N=12 lost one point on the same question, P1, which asks for my name and team. The answer gave the team but not the name, because the name wasn't in the top 3 facts. N=12 still got it right from its session summary. With one run and five questions, a one-question difference is a small signal.

4. Memory-saving policy (eviction)

My policy keeps the 4 newest messages in full. For anything older it drops the assistant's replies and keeps the user's own messages, because that is where the facts are.

Compared with N=12	N=12	Eviction	Change
Prompt tokens per turn	544.8	347.6	36% fewer

Latency (ms)	11,584	9,209	20% faster
Probe quality (1-5)	5.0	4.6	one question missed (P1)
Cost of 10,000 conversations	$387.62	$311.19	20% cheaper

5. Estimate memory cost

I used $1 per million input tokens and $5 per million output tokens, for 10,000 conversations of 30 turns. For each turn, the input is the prompt plus the extra memory calls, and the output is the reply plus what those calls write back.

Averaging the runs and working out the cost (code/hw6_sweep.py)

```python
def summarize(results):
    rows = list(csv.DictReader(open(TURNS_CSV)))
    summary = {}
    for label in SETTINGS:
        if label not in results:
            continue
        mine = [r for r in rows if r["user_id"] == f"sweep-{label}"][-30:]  # the latest complete run of this setting
        num = lambda key: [float(r[key]) for r in mine]
        own = [sum(float(r[k]) for k in ("primer_tokens", "long_term_tokens", "short_term_tokens", "current_tokens", "episodic_tokens")) for r in mine]
        extra_in = statistics.mean(num("extract_prompt_tokens")) + statistics.mean(num("summary_prompt_tokens"))
        extra_out = statistics.mean(num("extract_completion_tokens")) + statistics.mean(num("summary_completion_tokens"))
        s = {"mean_prompt_tokens_model": round(statistics.mean(num("prompt_tokens")), 1), "mean_prompt_tokens_own_count": round(statistics.mean(own), 1),
             "mean_completion_tokens": round(statistics.mean(num("completion_tokens")), 1), "mean_latency_ms": round(statistics.mean(num("latency_ms"))),
             "mean_reply_latency_ms": round(statistics.mean(num("reply_latency_ms"))), "maintenance_in_tokens_per_turn": round(extra_in, 1),
             "maintenance_out_tokens_per_turn": round(extra_out, 1), "mean_probe_quality": results[label]["mean_probe_quality"]}
        total_in = s["mean_prompt_tokens_model"] + extra_in
        total_out = s["mean_completion_tokens"] + extra_out
        s["cost_10000_convs_usd"] = round(10000 * 30 * (total_in * 1.0 + total_out * 5.0) / 1e6, 2)  # $1 per M input, $5 per M output
        s["cost_chat_only_usd"] = round(10000 * 30 * (s["mean_prompt_tokens_model"] * 1.0 + s["mean_completion_tokens"] * 5.0) / 1e6, 2)
        summary[label] = s
    json.dump(summary, open(RAW / "sweep_summary.json", "w"), indent=2)
    print("\n| setting | mean prompt tokens/turn (model) | (own count) | mean latency ms | probe quality 1-5 | cost 10k convs (chat only) | cost (with memory upkeep) |")
    print("|---|---|---|---|---|---|---|")
    for label, s in summary.items():
        print(f"| {label} | {s['mean_prompt_tokens_model']} | {s['mean_prompt_tokens_own_count']} | {s['mean_latency_ms']} | {s['mean_probe_quality']} | ${s['cost_chat_only_usd']} | ${s['cost_10000_convs_usd']} |")
```

Setting	Input tokens	Output tokens	Chat only	With memory upkeep
SHORT_TERM_N=4	223.8 M	21.5 M	$163.41	$331.18
SHORT_TERM_N=12 (best)	278.0 M	21.9 M	$215.49	$387.62
SHORT_TERM_N=30 (worst)	390.6 M	23.3 M	$340.41	$506.88
Eviction policy	215.0 M	19.2 M	$153.33	$311.19

How I picked the best and worst: I looked at answer quality first and cost second. N=12 is the only setting that got all five questions right, so it's the best, at $387.62. The other three tied at 4.6, and N=30 costs the most of those, at $506.88, which makes it the worst. If saving money matters more than that last question, the eviction policy is the best deal, about 20% cheaper than N=12. The extra memory calls make up about half of the cost in every setting, so a cheaper fact-extraction prompt would save more than changing the window size.

Part 4 - File-Based Memory and Consolidation

1. AGENT.md and MEMORY.md

AGENT.md started in HW1 and holds the long-term instructions for the assistant. MEMORY.md holds facts learned from earlier chats, one per line as "- [date] key: value", with a last_consolidated time at the top. My final AGENT.md has 16 instructions across all 5 categories: project description, technology stack, coding conventions, testing and prohibited actions. It also keeps the code-reviewer role from HW1.

Final AGENT.md

AGENT.md > abc # AGENT.md > abc ## Role: code reviewer

```
# AGENT.md

Durable instructions for the assistant working in this repo (DATA 260, SID4 3319, domain 7: Community Sports League Fixtures).

## Project description
- This repo is a community sports league fixtures app that grows from HW1 to HW6. Extend the existing code; do not rewrite it.
- The backend is a FastAPI service on port 8619 with a chat endpoint that uses short-term, summary and episodic memory.

## Technology stack
- Use Python 3.12, FastAPI, PyMongo with MongoDB 7 (Docker container s3319_mongo), and Ollama with qwen2.5:3b.
- Embeddings come from all-MiniLM-L6-v2. Names and prefixes use s3319; the random seed is 3319 and the verify seed is 263319.

## Coding conventions
- Use snake_case for functions and variables and UPPER_CASE for constants.
- Write only single-line `#` comments, each under 80 characters. Do not write docstrings.
- Keep every function under 40 lines and give it one job.

## Testing approach
- Every feature needs a deterministic check that runs with `make verify-hwNN`, using temperature 0 and the fixed seeds.
- A verification script may read the app and call its endpoints, but it must never modify app code.

## Prohibited actions
- Never hardcode passwords, tokens or connection secrets; read them from environment variables.
- Never use `eval`, `exec` or a bare `except:`.
- Never delete a fact from MEMORY.md silently; a removed or replaced fact must be recorded in the consolidation log or in a "supersedes" note.

## Role: code reviewer
- Use this role only when the user asks for a code review. For every other message, answer in short plain sentences.
- When reviewing code, reply only with bullet points, and start every line with "- ".
- When reviewing code, do not write paragraphs, introductions or closing remarks, and do not use code blocks or backticks.
- When reviewing code, keep each bullet short and specific, and review only the code or text the user gives.
```

2. File-based memory functions

code/file_memory.py loads both files and adds them to the system prompt. It reads the last_consolidated time and decides when a clean-up is needed: when MEMORY.md has more than 40 real lines, or when the last clean-up was more than 24 hours ago.

Loading the files, building the prompt and checking if consolidation is needed (code/file_memory.py)

code > file_memory.py > parse_memory

```python
# File memory: load AGENT.md and MEMORY.md, build the prompt, consolidate
import argparse
import json
import re
from datetime import datetime, timedelta
from pathlib import Path
REPO = Path(__file__).resolve().parent.parent
AGENT_PATH = REPO / "AGENT.md"
MEMORY_PATH = REPO / "MEMORY.md"
LOG_PATH = REPO / "MEMORY_CONSOLIDATION_LOG.md"
MAX_LINES = 40  # consolidate above this many content lines
MAX_AGE_HOURS = 24  # or when older than this
DREAM_HEADING = "## Recent (from /dream)"
STAMP_RE = re.compile(r"last_consolidated:\s*(\S+)")
ENTRY_RE = re.compile(r"^- \[(\d{4}-\d{2}-\d{2})\] ([^:]+): (.+)$")
SUPERSEDES_RE = re.compile(r"^(.*?)(?: \((supersedes .*)\))?$")
HEDGES = ("maybe", "might", "unconfirmed", "tentative", "not sure", "rumour", "rumor", "possibly")
def load_text(path):  # load one Markdown file, empty if missing
    path = Path(path)
    return path.read_text() if path.exists() else ""
def load_agent(path=AGENT_PATH):
    return load_text(path)
def load_memory(path=MEMORY_PATH):
    return load_text(path)
def content_lines(text):  # non-empty lines that are not HTML comments
    return [ln for ln in text.splitlines() if ln.strip() and not ln.strip().startswith("<!--")]
def plain_fact(line):
    match = ENTRY_RE.match(line)
    return f"{match.group(2)}: {match.group(3)}" if match else line.lstrip("# ").strip()
def build_system_prompt(agent_text, memory_text):  # AGENT.md, then MEMORY.md facts
    facts = "\n".join(plain_fact(ln) for ln in content_lines(memory_text))
    note = ("Use these facts when they help. Answer in short plain sentences, not as a copy of this list. "
            "You cannot change files or settings, so never say you updated or saved anything.")
    return f"{agent_text.strip()}\n\n# Long-term memory (from MEMORY.md)\n{note}\n{facts}"
def read_last_consolidated(text):  # last_consolidated timestamp or None
    match = STAMP_RE.search(text)
    if not match:
        return None
    try:
        return datetime.fromisoformat(match.group(1).rstrip("-> "))
    except ValueError:
        return None
def needs_consolidation(text, now=None):  # over 40 content lines or over 24h old
    now = now or datetime.now()
    count = len(content_lines(text))
    if count > MAX_LINES:
        return True, f"{count} content lines > {MAX_LINES}"
    stamp = read_last_consolidated(text)
    if stamp is None:
        return True, "no last_consolidated timestamp"
    if now - stamp > timedelta(hours=MAX_AGE_HOURS):
        return True, f"last consolidated {stamp.isoformat()} is over {MAX_AGE_HOURS}h old"
    return False, f"{count} content lines, consolidated {stamp.isoformat()}"
```

3. Memory commands

code/hw6_memory_cli.py is a small chat program. It rebuilds the system prompt from the two files on every turn. It has four commands: /memory shows what is in the files, /compact cleans up

MEMORY.md and shortens the older chat (it prints the counts before and after), /dream picks up lasting facts from recent messages and adds them to MEMORY.md, and /help lists the commands.

/help, /memory and /compact (code/hw6_memory_cli.py)

```
HELP = """Commands:
  /memory   show AGENT.md and MEMORY.md as the model sees them
  /compact  consolidate MEMORY.md and compress older chat messages (counts before and after)
  /dream    pull lasting facts out of the recent chat and add them to MEMORY.md
  /help     show this list
  /quit     leave"""
def make_client():
    return ModelClient(num_ctx=4096, num_predict=200, seed=3319)
def system_message(agent_path, memory_path):
    return {"role": "system", "content": fm.build_system_prompt(fm.load_agent(agent_path), fm.load_memory(memory_path))}
def show_memory(agent_path, memory_path):
    agent, mem = fm.load_agent(agent_path), fm.load_memory(memory_path)
    due, why = fm.needs_consolidation(mem)
    print(f"AGENT.md: {len(fm.content_lines(agent))} content lines")
    print(f"MEMORY.md: {len(fm.content_lines(mem))} content lines, last consolidated {fm.read_last_consolidated(mem)}")
    print(f"consolidation required: {due} ({why})")
    print("--- MEMORY.md ---")
    print(mem.rstrip())
def compact(history, client, memory_path):  # /compact: report counts before and after
    before_msgs, lines_before = len(history), len(fm.content_lines(fm.load_memory(memory_path)))
    _, lines_after, log = fm.run_consolidation(memory_path)
    if len(history) > KEEP_RECENT:
        old, recent = history[:-KEEP_RECENT], history[-KEEP_RECENT:]
        text = "\n".join(f"{m['role']}: {m['content']}" for m in old)
        ask = [{"role": "system", "content": "Summarize this chat in 2 short bullets starting with '- '."},
               {"role": "user", "content": text}]
        summary = client.complete(ask, json_mode=False)["text"].strip()
        history[:] = [{"role": "assistant", "content": "Summary of earlier chat:\n" + summary}] + recent
    print(f"messages: {before_msgs} -> {len(history)}")
    print(f"MEMORY.md content lines: {lines_before} -> {lines_after} ({len(log)} changes logged in {fm.LOG_PATH.name})")
KEY_HINTS = {"name": "the user's name", "role": "the user's job", "team": "the team's name",
             "home venue": "where home games are played", "away venue": "where away games are played",
             "rival": "the opposing team", "next game": "date and time of the next game",
             "training day": "the day the team trains", "reminders": "how long before games to send reminders",
             "availability": "days and times the user is free", "timezone": "the user's time zone",
             "contact": "how to contact the user", "summary style": "how summaries should look"}
def dream_prompt(keys):
    shown = ", ".join(f"{k} ({KEY_HINTS[k]})" if k in KEY_HINTS else k for k in keys)
    return ("Read the user's messages and list lasting facts they state about themselves, their team, venue, schedule or "
            f"preferences. Reuse one of these keys only when its meaning fits the fact: {shown}. Otherwise use a new key of "
            "one to three words. Copy each value from the user's own words. Reply with JSON only. Examples: "
            "'I can only help on Tuesdays' -> {\"facts\": [{\"key\": \"availability\", \"value\": \"Tuesdays only\"}]}; "
            "'Please remind me a day before each game' -> {\"facts\": [{\"key\": \"reminders\", \"value\": \"a day before each game\"}]}. "
            'Ignore questions and small talk. If nothing lasting was said, reply {"facts": []}.')
```

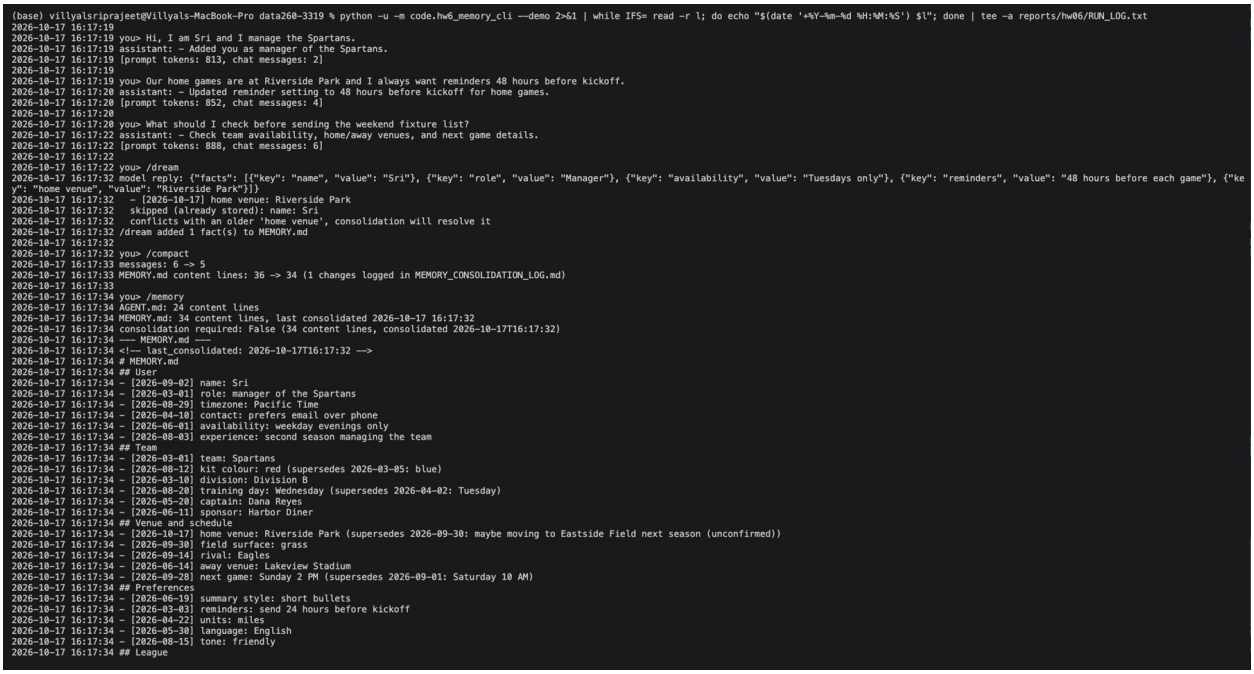
/dream, a chat turn, and how each command is handled (code/hw6_memory_cli.py)

```python
def dream(history, client, memory_path):  # /dream: add lasting facts to MEMORY.md
    mem = fm.load_memory(memory_path)
    users = [m["content"] for m in history[-10:] if m["role"] == "user"]
    said = "\n".join(t for t in users if not t.strip().endswith("?"))  # questions are not facts
    ask = [{"role": "system", "content": dream_prompt(fm.memory_keys(mem))}, {"role": "user", "content": said}]
    raw = client.complete(ask, json_mode=True)["text"]
    print("model reply:", raw.strip())
    new_text, added, notes = fm.add_facts(mem, fm.parse_dream(raw, said))
    Path(memory_path).write_text(new_text)
    for line in added + notes:
        print(" ", line)
    print(f"/dream added {len(added)} fact(s) to MEMORY.md")
def reply(history, client, user_text, agent_path, memory_path):  # system prompt rebuilt from the files each turn
    history.append({"role": "user", "content": user_text})
    messages = [system_message(agent_path, memory_path)] + history
    out = client.complete(messages, json_mode=False)
    history.append({"role": "assistant", "content": out["text"].strip()})
    return out["text"].strip(), out["input_tokens"]
def handle(line, history, client, agent_path, memory_path):  # route a command or a chat line
    if line == "/quit":
        return False
    if line == "/help":
        print(HELP)
    elif line == "/memory":
        show_memory(agent_path, memory_path)
    elif line == "/compact":
        compact(history, client, memory_path)
    elif line == "/dream":
        dream(history, client, memory_path)
    else:
        text, tokens = reply(history, client, line, agent_path, memory_path)
        print(f"assistant: {text}\n[prompt tokens: {tokens}, chat messages: {len(history)}]")
    return True
```

Demo session - /help and /memory (34 lines, no clean-up needed yet)

```
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python -u -m code.hw6_memory_cli --demo 2>&1 | while IFS= read -r l; do echo "$(date '+%Y-%m-%d %H:%M:%S') $l"; done | tee -a reports/hw06/RUN_LOG.txt
2026-10-17 16:17:18 startup: consolidation required = False (34 content lines, consolidated 2026-10-17T16:00:51)
2026-10-17 16:17:18
2026-10-17 16:17:18 you> /help
2026-10-17 16:17:18 Commands:
2026-10-17 16:17:18   /memory   show AGENT.md and MEMORY.md as the model sees them
2026-10-17 16:17:18   /compact  consolidate MEMORY.md and compress older chat messages (counts before and after)
2026-10-17 16:17:18   /dream    pull lasting facts out of the recent chat and add them to MEMORY.md
2026-10-17 16:17:18   /help     show this list
2026-10-17 16:17:18   /quit     leave
2026-10-17 16:17:18
2026-10-17 16:17:19 you> /memory
2026-10-17 16:17:19 AGENT.md: 24 content lines
2026-10-17 16:17:19 MEMORY.md: 34 content lines, last consolidated 2026-10-17 16:00:51
2026-10-17 16:17:19 consolidation required: False (34 content lines, consolidated 2026-10-17T16:00:51)
2026-10-17 16:17:19 --- MEMORY.md ---
2026-10-17 16:17:19 <!-- last_consolidated: 2026-10-17T16:00:51 -->
2026-10-17 16:17:19 # MEMORY.md
2026-10-17 16:17:19 ## User
2026-10-17 16:17:19 - [2026-09-02] name: Sri
2026-10-17 16:17:19 - [2026-03-01] role: manager of the Spartans
2026-10-17 16:17:19 - [2026-08-29] timezone: Pacific Time
2026-10-17 16:17:19 - [2026-04-10] contact: prefers email over phone
2026-10-17 16:17:19 - [2026-06-01] availability: weekday evenings only
2026-10-17 16:17:19 - [2026-08-03] experience: second season managing the team
2026-10-17 16:17:19 ## Team
2026-10-17 16:17:19 - [2026-03-01] team: Spartans
2026-10-17 16:17:19 - [2026-08-12] kit colour: red (supersedes 2026-03-05: blue)
2026-10-17 16:17:19 - [2026-03-10] division: Division B
2026-10-17 16:17:19 - [2026-08-20] training day: Wednesday (supersedes 2026-04-02: Tuesday)
2026-10-17 16:17:19 - [2026-05-20] captain: Dana Reyes
2026-10-17 16:17:19 - [2026-06-11] sponsor: Harbor Diner
2026-10-17 16:17:19 ## Venue and schedule
2026-10-17 16:17:19 - [2026-09-30] home venue: maybe moving to Eastside Field next season (unconfirmed) (supersedes 2026-03-01: Riverside Park)
2026-10-17 16:17:19 - [2026-09-30] field surface: grass
2026-10-17 16:17:19 - [2026-09-14] rival: Eagles
2026-10-17 16:17:19 - [2026-06-14] away venue: Lakeview Stadium
2026-10-17 16:17:19 - [2026-09-28] next game: Sunday 2 PM (supersedes 2026-09-01: Saturday 10 AM)
2026-10-17 16:17:19 ## Preferences
2026-10-17 16:17:19 - [2026-06-19] summary style: short bullets
2026-10-17 16:17:19 - [2026-03-03] reminders: send 24 hours before kickoff
2026-10-17 16:17:19 - [2026-04-22] units: miles
2026-10-17 16:17:19 - [2026-05-30] language: English
2026-10-17 16:17:19 - [2026-08-15] tone: friendly
2026-10-17 16:17:19 ## League
2026-10-17 16:17:19 - [2026-09-09] league: Community Sports League
2026-10-17 16:17:19 - [2026-07-30] points for win: 3
2026-10-17 16:17:19 - [2026-03-01] points for draw: 1
2026-10-17 16:17:19 - [2026-04-05] season length: 12 weeks
2026-10-17 16:17:19 - [2026-06-21] fixture source: league website
2026-10-17 16:17:19 - [2026-08-25] rules: roster locks after week 3
2026-10-17 16:17:19
2026-10-17 16:17:19 you> Hi, I am Sri and I manage the Spartans.
2026-10-17 16:17:19 assistant: - Added you as manager of the Spartans.
2026-10-17 16:17:19 [prompt tokens: 813, chat messages: 2]
2026-10-17 16:17:19
```

/dream adds "home venue: Riverside Park"; /compact goes from 6 to 5 messages and 36 to 34 lines



The small 3B model sometimes starts a reply with "- " or says it saved something, even though the prompt tells it that it can't change files. Only /dream and /compact actually write to MEMORY.md.

4. Verifying a generated function against AGENT.md

I checked needs_consolidation, and the other functions in the two Part 4 files, with code/check_agent_rules.py. It reads AGENT.md and checks the code against its rules: snake_case names, no docstrings, every function under 40 lines, short one-line comments, no eval, exec or bare except, no passwords in the code, and UPPER_CASE constants. It also tests needs_consolidation itself, including the edge case of exactly 40 lines.

The rule checks (code/check_agent_rules.py)

```python
def check_functions(tree, label):  # naming, docstrings, length, eval/exec, bare except
    funcs = [n for n in ast.walk(tree) if isinstance(n, ast.FunctionDef)]
    bad_names = [f.name for f in funcs if not re.fullmatch(r"[a-z_][a-z0-9_]*", f.name)]
    record(f"{label}: function names are snake_case", not bad_names, str(bad_names or ""))
    docs = [f.name for f in funcs if ast.get_docstring(f)]
    record(f"{label}: no docstrings", not docs, str(docs or ""))
    long = [f.name for f in funcs if f.end_lineno - f.lineno + 1 >= 40]
    record(f"{label}: every function under 40 lines", not long, str(long or ""))
    calls = [n.func.id for n in ast.walk(tree) if isinstance(n, ast.Call) and isinstance(n.func, ast.Name)]
    record(f"{label}: no eval or exec", not {"eval", "exec"} & set(calls))
    bare = [n for n in ast.walk(tree) if isinstance(n, ast.ExceptHandler) and n.type is None]
    record(f"{label}: no bare except", not bare)
def check_constants_and_secrets(tree, label):  # UPPER_CASE constants, no hardcoded secrets
    names = [t.id for n in tree.body if isinstance(n, ast.Assign) for t in n.targets if isinstance(t, ast.Name)]
    bad = [n for n in names if not n.isupper()]
    record(f"{label}: module constants are UPPER_CASE", not bad, str(bad or ""))
    leaks = [t.id for n in ast.walk(tree) if isinstance(n, ast.Assign) and isinstance(n.value, ast.Constant)
             and isinstance(n.value.value, str) and n.value.value for t in n.targets
             if isinstance(t, ast.Name) and any(w in t.id.lower() for w in SECRET_WORDS)]
    record(f"{label}: no hardcoded secrets", not leaks, str(leaks or ""))
def check_comments(source, label):  # one-line comments under 80 characters
    comments = [t.string for t in tokenize.generate_tokens(io.StringIO(source).readline) if t.type == tokenize.COMMENT]
    long = [c for c in comments if len(c) >= 80]
    record(f"{label}: comments are one line under 80 characters", not long, f"({len(comments)} comments)")
def check_behaviour():  # 40-line and 24-hour rules
    import file_memory as fm
    now = datetime(2026, 10, 17, 12, 0, 0)
    fresh = (now - timedelta(hours=1)).isoformat()
    old = (now - timedelta(hours=25)).isoformat()
    body = "\n".join(f"- [2026-01-01] k{i}: v" for i in range(41))
    short = "\n".join(f"- [2026-01-01] k{i}: v" for i in range(5))
    cases = [(f"<!-- last_consolidated: {fresh} -->\n{short}", False), (f"<!-- last_consolidated: {old} -->\n{short}", True),
             (f"<!-- last_consolidated: {fresh} -->\n{body}", True), (short, True)]
    got = [fm.needs_consolidation(text, now)[0] for text, _ in cases]
    record("needs_consolidation: fresh and short -> no, old -> yes, 41 lines -> yes, no stamp -> yes",
           got == [want for _, want in cases], str(got))
    forty = "\n".join(f"- [2026-01-01] k{i}: v" for i in range(40))
    edge = fm.needs_consolidation(f"<!-- last_consolidated: {fresh} -->\n{forty}", now)[0]
    record("needs_consolidation: exactly 40 lines is not over the limit", edge is False)
```

python -m code.check_agent_rules - 20 of 20 checks passed

```
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % cd ~/Desktop/data260-3319

(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python -m code.check_agent_rules
  [PASS] AGENT.md has at least 3 categories (5: Project description, Technology stack, Coding conventions, Testing approach, Prohibited actions)
  [PASS] AGENT.md has at least 5 instructions (16 bullets)
  [PASS] file_memory.py: function names are snake_case
  [PASS] file_memory.py: no docstrings
  [PASS] file_memory.py: every function under 40 lines
  [PASS] file_memory.py: no eval or exec
  [PASS] file_memory.py: no bare except
  [PASS] file_memory.py: module constants are UPPER_CASE
  [PASS] file_memory.py: no hardcoded secrets
  [PASS] file_memory.py: comments are one line under 80 characters (22 comments)
  [PASS] hw6_memory_cli.py: function names are snake_case
  [PASS] hw6_memory_cli.py: no docstrings
  [PASS] hw6_memory_cli.py: every function under 40 lines
  [PASS] hw6_memory_cli.py: no eval or exec
  [PASS] hw6_memory_cli.py: no bare except
  [PASS] hw6_memory_cli.py: module constants are UPPER_CASE
  [PASS] hw6_memory_cli.py: no hardcoded secrets
  [PASS] hw6_memory_cli.py: comments are one line under 80 characters (11 comments)
  [PASS] needs_consolidation: fresh and short -> no, old -> yes, 41 lines -> yes, no stamp -> yes [False, True, True, True]
  [PASS] needs_consolidation: exactly 40 lines is not over the limit
  20/20 checks passed
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

5. Consolidation test

I wrote a 55-line MEMORY.md with 7 repeated facts (some only differ in capitals or a full stop) and 4 facts that contradict each other. The clean-up groups lines by key. Repeats are merged into one line. When two values disagree, the newest one wins, and the older one stays as a "supersedes" note so nothing quietly disappears. Every change is written to MEMORY_CONSOLIDATION_LOG.md. If the winning value sounds unsure ("maybe", "unconfirmed"), it is marked REVIEW, because newest-wins could be wrong there.

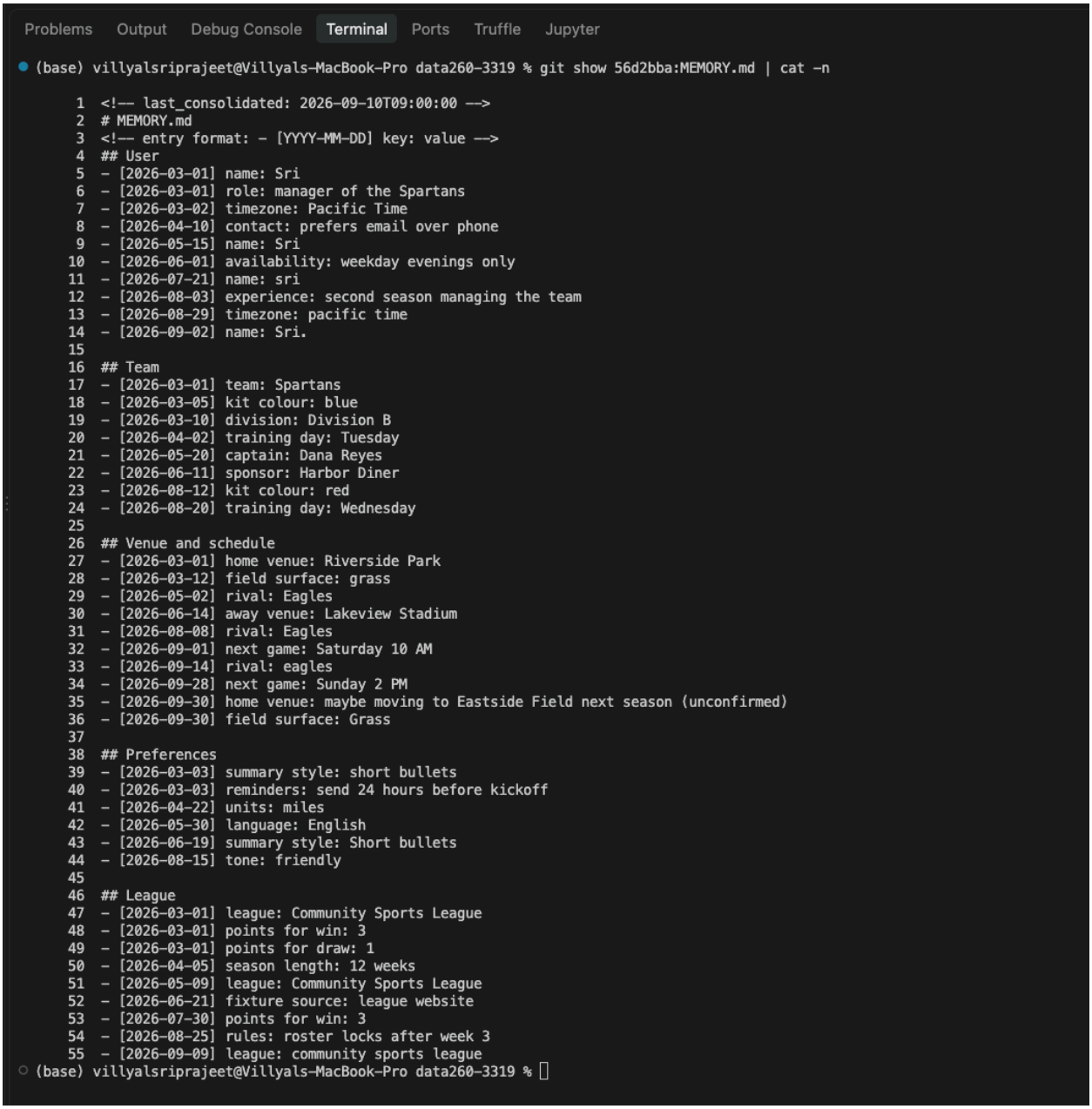
Merging repeats and settling contradictions (code/file_memory.py)

```python
def resolve_group(group, log):  # merge duplicates or resolve a contradiction
    ranked = sorted(enumerate(group), key=lambda p: (p[1]["date"], p[0]))
    winner = dict(ranked[-1][1])
    key = winner["key"]
    values = {normalise(e["value"]) for e in group}
    notes = [p for _, e in ranked if e["history"] for p in e["history"].split("; ")]
    if len(values) == 1:
        winner["value"] = group[0]["value"]
        if len(group) > 1:
            dates = ", ".join(e["date"] for e in group)
            log.append(f"MERGED duplicate | {key} | {len(group)} copies ({dates}) -> kept {winner['date']}")
        winner["history"] = clean_history(notes, winner["value"])
        return winner
    old = {}
    for _, e in ranked[:-1]:
        if normalise(e["value"]) != normalise(winner["value"]):
            old[normalise(e["value"])] = e
    shown = [f"supersedes {e['date']}: {e['value']}" for e in old.values()]
    winner["history"] = clean_history(notes + shown, winner["value"])
    log.append(f"CONTRADICTION | {key} | kept {winner['date']} '{winner['value']}' over " + "; ".join(f"{e['date']} '{e['value']}'" for e in old.values()))
    if any(h in winner["value"].lower() for h in HEDGES):
        log.append(f"REVIEW | {key} | newest value is hedged, so newest-wins may be WRONG; the older value may still be true")
    return winner
def format_entry(entry):
    if "raw" in entry:
        return entry["raw"]
    tail = f" ({entry['history']})" if entry["history"] else ""
    return f"- [{entry['date']}] {entry['key']}: {entry['value']}{tail}"
```

Running the clean-up and writing the log (code/file_memory.py)

```python
def consolidate(text, now=None):  # newest wins, every change is logged
    now = now or datetime.now()
    title, sections = parse_memory(text)
    groups, placed = {}, {heading: [] for heading in sections}
    for heading, items in sections.items():
        for item in items:
            gid = ("raw", item["raw"]) if "raw" in item else normalise(item["key"])
            if gid not in groups:
                groups[gid] = []
                placed[heading].append(gid)
            groups[gid].append(item)
    out, log = [f"<!-- last_consolidated: {now.replace(microsecond=0).isoformat()} -->", title], []
    for heading, gids in placed.items():
        if heading and gids:
            out.append(heading)
        for gid in gids:
            grp = groups[gid]
            out.append(format_entry(grp[0] if "raw" in grp[0] else resolve_group(grp, log)))
    return "\n".join(out) + "\n", log
def write_log(log, before, after, path=LOG_PATH, now=None):  # audit log, nothing removed silently
    now = now or datetime.now()
    lines = [f"# Consolidation log ({now.replace(microsecond=0).isoformat()})", "",
             f"Content lines before: {before}. After: {after}.", ""]
    lines += [f"- {entry}" for entry in log] or ["- no changes"]
    Path(path).write_text("\n".join(lines) + "\n")
def run_consolidation(path=MEMORY_PATH, log_path=LOG_PATH):  # returns (before, after, log)
    text = load_memory(path)
    new_text, log = consolidate(text)
    before, after = len(content_lines(text)), len(content_lines(new_text))
    Path(path).write_text(new_text)
    write_log(log, before, after, log_path)
    return before, after, log
```

Original MEMORY.md (commit 56d2bba, 55 lines) and after consolidation (commit d9febe7, 35 lines)

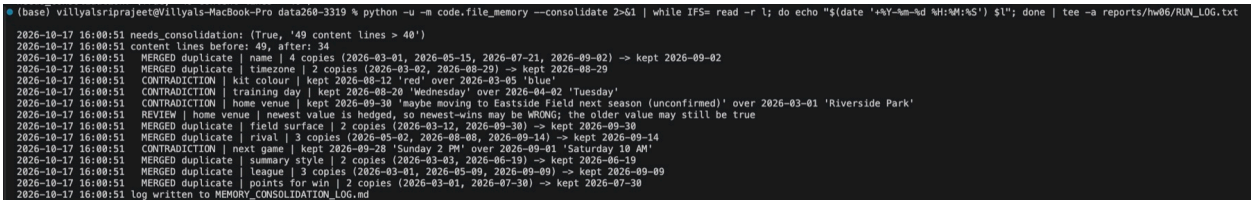


```
Problems  Output  Debug Console  Terminal  Ports  Truffle  Jupyter

(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % git show d9febe7:MEMORY.md | cat -n

     1  <!-- last_consolidated: 2026-10-17T19:58:40 -->
     2  # MEMORY.md
     3  ## User
     4  - [2026-09-02] name: Sri
     5  - [2026-03-01] role: manager of the Spartans
     6  - [2026-08-29] timezone: Pacific Time
     7  - [2026-04-10] contact: prefers email over phone
     8  - [2026-06-01] availability: weekday evenings only
     9  - [2026-08-03] experience: second season managing the team
    10  ## Team
    11  - [2026-03-01] team: Spartans
    12  - [2026-08-12] kit colour: red (supersedes 2026-03-05: blue)
    13  - [2026-03-10] division: Division B
    14  - [2026-08-20] training day: Wednesday (supersedes 2026-04-02: Tuesday)
    15  - [2026-05-20] captain: Dana Reyes
    16  - [2026-06-11] sponsor: Harbor Diner
    17  ## Venue and schedule
    18  - [2026-09-30] home venue: maybe moving to Eastside Field next season (unconfirmed) (supersedes 2026-03-01: Riverside Park)
    19  - [2026-09-30] field surface: grass
    20  - [2026-09-14] rival: Eagles
    21  - [2026-06-14] away venue: Lakeview Stadium
    22  - [2026-09-28] next game: Sunday 2 PM (supersedes 2026-09-01: Saturday 10 AM)
    23  ## Preferences
    24  - [2026-06-19] summary style: short bullets
    25  - [2026-03-03] reminders: send 24 hours before kickoff
    26  - [2026-04-22] units: miles
    27  - [2026-05-30] language: English
    28  - [2026-08-15] tone: friendly
    29  ## League
    30  - [2026-09-09] league: Community Sports League
    31  - [2026-07-30] points for win: 3
    32  - [2026-03-01] points for draw: 1
    33  - [2026-04-05] season length: 12 weeks
    34  - [2026-06-21] fixture source: league website
    35  - [2026-08-25] rules: roster locks after week 3
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

The clean-up run - 49 to 34 lines, 7 merges, 4 contradictions, 1 REVIEW flag



```
(base) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python -u -m code.file_memory --consolidate 2>&1 | while IFS= read -r l; do echo "$(date '+%Y-%m-%d %H:%M:%S') $l"; done | tee -a reports/hw06/RUN_LOG.txt

2026-10-17 16:00:51 needs_consolidation: (True, '49 content lines > 40')
2026-10-17 16:00:51 content lines before: 49, after: 34
2026-10-17 16:00:51   MERGED duplicate | name | 4 copies (2026-03-01, 2026-05-15, 2026-07-21, 2026-09-02) -> kept 2026-09-02
2026-10-17 16:00:51   MERGED duplicate | timezone | 2 copies (2026-03-02, 2026-08-29) -> kept 2026-08-29
2026-10-17 16:00:51   CONTRADICTION | kit colour | kept 2026-08-12 'red' over 2026-03-05 'blue'
2026-10-17 16:00:51   CONTRADICTION | training day | kept 2026-08-20 'Wednesday' over 2026-04-02 'Tuesday'
2026-10-17 16:00:51   CONTRADICTION | home venue | kept 2026-09-30 'maybe moving to Eastside Field next season (unconfirmed)' over 2026-03-01 'Riverside Park'
2026-10-17 16:00:51   REVIEW | home venue | newest value is hedged, so newest-wins may be WRONG; the older value may still be true
2026-10-17 16:00:51   MERGED duplicate | field surface | 2 copies (2026-03-12, 2026-09-30) -> kept 2026-09-30
2026-10-17 16:00:51   MERGED duplicate | rival | 3 copies (2026-05-02, 2026-08-08, 2026-09-14) -> kept 2026-09-14
2026-10-17 16:00:51   CONTRADICTION | next game | kept 2026-09-28 'Sunday 2 PM' over 2026-09-01 'Saturday 10 AM'
2026-10-17 16:00:51   MERGED duplicate | summary style | 2 copies (2026-03-03, 2026-06-19) -> kept 2026-06-19
2026-10-17 16:00:51   MERGED duplicate | league | 3 copies (2026-03-01, 2026-05-09, 2026-09-09) -> kept 2026-09-09
2026-10-17 16:00:51   MERGED duplicate | points for win | 2 copies (2026-03-01, 2026-07-30) -> kept 2026-07-30
2026-10-17 16:00:51 log written to MEMORY_CONSOLIDATION_LOG.md
```

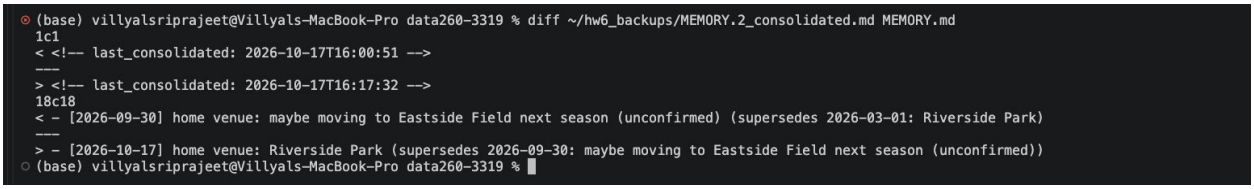
MEMORY_CONSOLIDATION_LOG.md

```
MEMORY_CONSOLIDATION_LOG.md > abc # Consolidation log (2026-10-17T16:00:51)
 1  # Consolidation log (2026-10-17T16:00:51)
 2
 3  Content lines before: 49. After: 34.
 4
 5  - MERGED duplicate | name | 4 copies (2026-03-01, 2026-05-15, 2026-07-21, 2026-09-02) - kept 2026-09-02
 6  - MERGED duplicate | timezone | 2 copies (2026-03-02, 2026-08-29) - kept 2026-08-29
 7  - CONTRADICTION | kit colour | kept 2026-08-12 'red' over 2026-03-05 'blue'
 8  - CONTRADICTION | training day | kept 2026-08-20 'Wednesday' over 2026-04-02 'Tuesday'
 9  - CONTRADICTION | home venue | kept 2026-09-30 'maybe moving to Eastside Field next season (unconfirmed)' over 2026-03-01 'Riverside Park'
10  - REVIEW | home venue | newest value is hedged, so newest-wins may be WRONG; the older value may still be true
11  - MERGED duplicate | field surface | 2 copies (2026-03-12, 2026-09-30) - kept 2026-09-30
12  - MERGED duplicate | rival | 3 copies (2026-05-02, 2026-08-08, 2026-09-14) - kept 2026-09-14
13  - CONTRADICTION | next game | kept 2026-09-28 'Sunday 2 PM' over 2026-09-01 'Saturday 10 AM'
14  - MERGED duplicate | summary style | 2 copies (2026-03-03, 2026-06-19) - kept 2026-06-19
15  - MERGED duplicate | league | 3 copies (2026-03-01, 2026-05-09, 2026-09-09) - kept 2026-09-09
16  - MERGED duplicate | points for win | 2 copies (2026-03-01, 2026-07-30) - kept 2026-07-30
17
```

Contradiction	Kept (newest)	Kept as a note	Right?
kit colour	red (2026-08-12)	blue (2026-03-05)	Yes
training day	Wednesday (2026-08-20)	Tuesday (2026-04-02)	Yes
next game	Sunday 2 PM (2026-09-28)	Saturday 10 AM (2026-09-01)	Yes
home venue	maybe moving to Eastside Field next season (unconfirmed)	Riverside Park (2026-03-01)	No - marked REVIEW

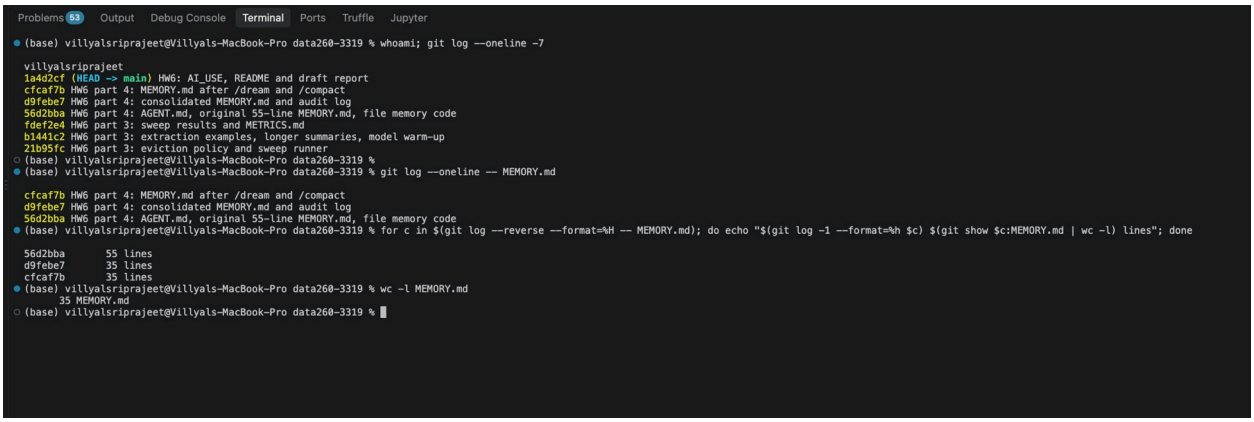
The one it got wrong: newest-wins swapped my confirmed home ground, Riverside Park, for an unconfirmed rumour about Eastside Field. The old value wasn't lost, though. It stayed as a note and the log marked it REVIEW. Later in the demo, /dream picked up "home venue: Riverside Park" from the chat, and the next /compact put it back, keeping the rumour as a note.

The change in MEMORY.md after /dream and /compact - Riverside Park is back



All three versions are in Git: the original (56d2bba, 55 lines), after consolidation (d9febe7, 35 lines), and after /dream and /compact (cfcaf7b, 35 lines).

Git history of MEMORY.md with the line count of each version



6. Comparing both memory systems

Criterion	MongoDB memory (Parts 2-3)	File-based memory (AGENT.md / MEMORY.md)
Inspectability	You need Compass or a query to look at it, and embeddings can't be read by a person	Plain text you can open in any editor, and every change shows up in Git
Persistence after a restart	Survives restarts through the Docker volume, but is lost if the volume is deleted	Saved on disk and in Git, so older versions can be brought back
Scalability across many sessions	Indexed by user and session, so each prompt only takes a small slice	The whole file goes into every prompt, so it has to be cleaned up and won't scale to many facts
Facts that must not be forgotten	Can slip: top-3 retrieval dropped my name in probe P1	Always in the prompt; the risk moves to the clean-up, which is why replaced facts keep a note

In practice I would keep chat history and saved facts in MongoDB, where they scale to many users, and keep the few facts that must never be lost in a short MEMORY.md that a person checks.

Verification (verification.json)

make verify-hw06 runs code/verify_hw06.py. It checks that the required files are there, the Part 3 data is complete, the MongoDB collections and their rules exist, the file memory and commands work, the AGENT.md checker passes, and the clean-up gives the right result when replayed on the committed 55-line file. Then it checks that Ollama has the model and calls the live API on port 8619. It only reads files and calls endpoints. It never changes the app's code.

The settings it records and the check helper (code/verify_hw06.py)

```
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "code"))
REP = ROOT / "reports" / "hw06"
RAW = REP / "raw"
SID4 = 3319
CONFIG = {"SID4": SID4, "PORT_BASE": 8000 + SID4 % 900, "PREFIX": f"s{SID4}", "SEED": SID4,
          "VERIFY_SEED": 260000 + SID4, "DOMAIN_ID": SID4 % 8}
BASE = f"http://localhost:{CONFIG['PORT_BASE']}"
MODEL = {"llm": "qwen2.5:3b", "runtime": "Ollama (local)", "temperature": 0, "seed": CONFIG["SEED"], "num_ctx": 4096,
         "database": "MongoDB 7.0 (Docker s3319_mongo)", "short_term_n": 12, "summarize_every_user_msgs": 5,
         "episodic_top_k": 3, "embedder": "all-MiniLM-L6-v2", "file_memory_max_lines": 40, "file_memory_max_age_h": 24}
checks = []
def check(name, ok, detail=""):
    checks.append({"check": name, "passed": bool(ok), "detail": str(detail)})
    print(("PASS" if ok else "FAIL"), "-", name, f"({detail})" if detail else "")
def port_open(port):
    with socket.socket() as s:
        return s.connect_ex(("127.0.0.1", port)) == 0
def git(*args):
    try:
        return subprocess.check_output(["git", *args], cwd=ROOT, stderr=subprocess.DEVNULL).decode().strip()
    except Exception:
        return ""
```

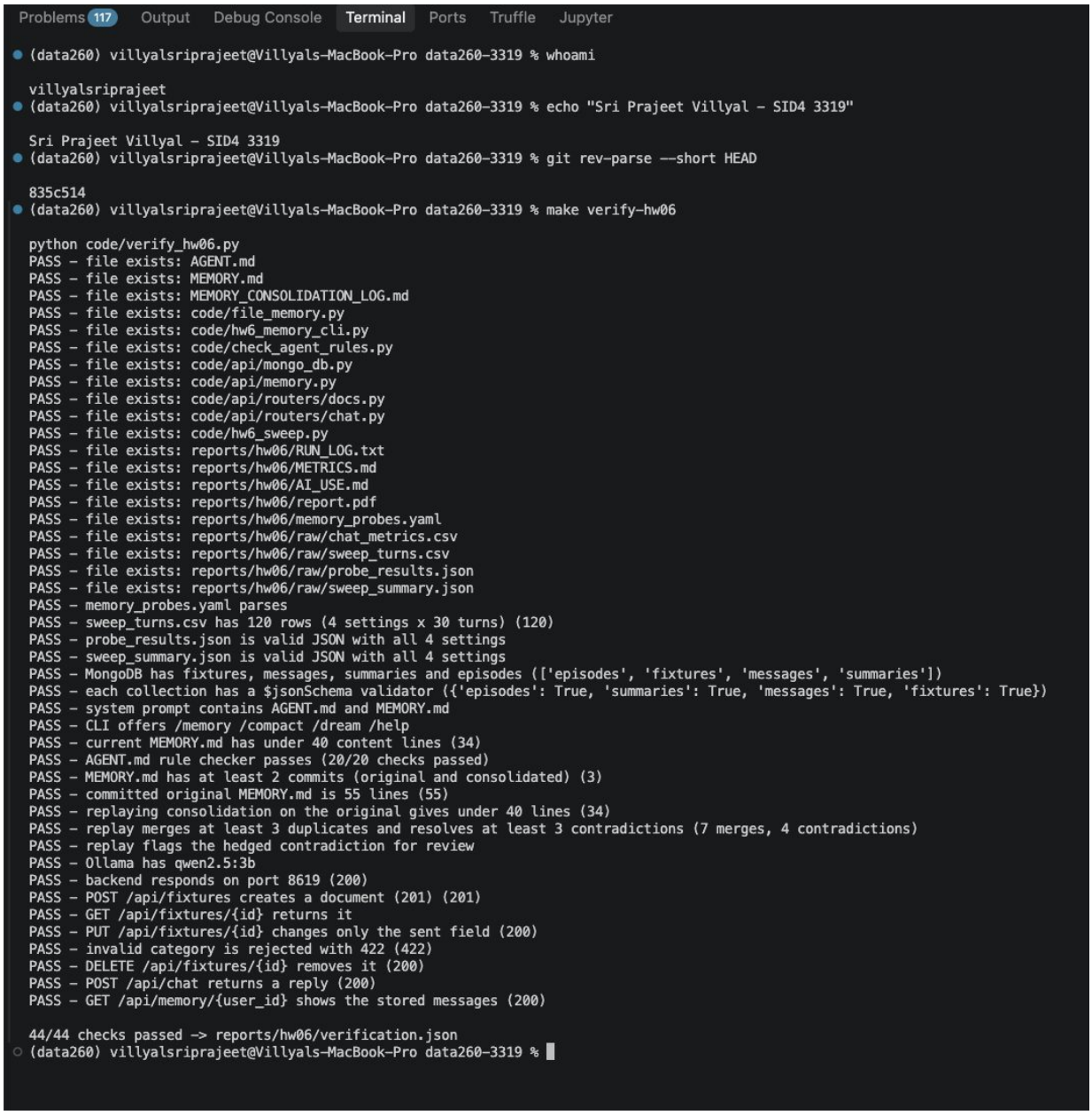
Writing verification.json (code/verify_hw06.py)

```
    check(f"backend responds on port {CONFIG['PORT_BASE']}", False, e)
finally:
    if server:
        server.terminate()
out = {"hw": "hw6", "sid4": SID4, "generated": datetime.now().isoformat(timespec="seconds"),
       "commit_hash": git("rev-parse", "HEAD") or "unknown", "tag": git("describe", "--tags", "--exact-match") or "none",
       "config": CONFIG, "model": MODEL, "all_passed": all(c["passed"] for c in checks), "checks": checks}
REP.mkdir(parents=True, exist_ok=True)
(REP / "verification.json").write_text(json.dumps(out, indent=2))
print(f"\n{sum(c['passed'] for c in checks)}/{len(checks)} checks passed -> reports/hw06/verification.json")

```

Field in verification.json	Value
hw	hw6
sid4	3319
commit_hash	835c514d1eef7b2babd15a8f1423bb48573dfaec
model	qwen2.5:3b on Ollama, temperature 0, seed 3319, num_ctx 4096, MongoDB 7.0, SHORT_TERM_N 12
SEED / VERIFY_SEED	3319 / 263319
checks	44 checks, each with passed true/false and a detail
all_passed	true (44 of 44)

make verify-hw06 on commit 835c514 - 44 of 44 checks passed



AI Use

1. What I used an AI assistant for and what I did myself

I used AI as an assistant for this homework. It wrote the first drafts of the Mongo routes, the chat memory, the sweep script, the file memory module and the report text, and it walked me through each step. I set up MongoDB, Ollama and the environment on my laptop, ran every command myself, took all the screenshots, and checked the results against Compass and the raw files. Every number in the report comes from my own runs.

2. One AI output that was wrong

The first fact extraction prompt didn't work well with qwen2.5:3b. In my first N=4 run it found nothing in four of the six messages that had facts, and one session summary got cut off, so the change to Sunday 2 PM was lost. That run scored 4.2, and the next-game question got 1 out of 5.

3. How I detected the problem

The score looked too low, so I printed the saved facts and summaries for that run. The fact messages had nothing saved, and one summary stopped halfway through a line. I kept the files from that first run so the difference can be seen.

4. What I changed and why it works now

I added three short examples to the extraction prompt, made the summaries stricter and gave them more room, and added a warm-up call before timing. A separate check then found all six facts and nothing in the filler messages, so I reran all four settings and used the new results. The same kind of checking helped in Part 4 too. The AGENT.md checker failed 2 of 20 checks because some comments were too long, so I shortened them. The first /dream runs saved a made up key and a question as a fact, so I added key hints, a rule that values must come from the user's own words, and a rule to skip questions.